Real-time Target Speaker Extraction

Grant Booysen

25849646@sun.ac.za

Report presented in partial fulfilment of the requirements for the degree of

MSc in Machine Learning & Artificial Intelligence

in the Faculty of Science at Stellenbosch University

Supervisors: Dewald J. Swanevelder and David Botha (Smart Operator)

November 2026

Abstract

This work builds a causal, real-time, single-channel target speaker extractor whose output is judged by a live speech-to-speech model, Gemini. This work analyses how to build an extractor whose output improves intelligibility for a live model, rather than optimising for signal separation quality. Two errors are considered: words leaked from the interfering speaker and words that neither speaker said that the live model invented or mis-transcribed. An evaluation suite is thus proposed to consider both errors and states the relationship between them. The extractor built is a causal band-split network, conditioned on an enrollment clip of the target speaker. The extractor is trained on a simulated dataset created from LibriSpeech speakers, with random room acoustics and background noise. Since the live model cannot be differentiated through, a proxy loss is used to train the extractor. Splitting the cue and adding a speaker embedding to the extractor showed improvement over the standard band separation architecture, where the Gemini word error rate on the test set dropped from 48.46 % for the baseline to 41.81 % for the extension, while cutting the leakage of the interfering speaker's words by about a third. The model is seen to perform in real time with a $175\,\mathrm{ms}$ latency on a laptop CPU. The extractor is evaluated on real recordings, holding the same conclusions as the simulated data. Perceptual and signal-domain metrics do not convincingly follow the ranking set out by Gemini. On the test set, the extension beats just passing the mixture straight to the live model by dropping the word error rate from 52.05 % to 41.81 %, the rate at which two or more words are leaked in a trial drops from 60.57 % to 32.93 %, the share of the interfering speaker's words reaching Gemini drops from 47.89 % to 21.15 %, but at the cost of an increased rate of two or more invented words from 30.24 % to 47.74 %. Finally, the extension is compared against an established offline system, WeSep, which is trained on different data and is not optimised for the live model. WeSep recovers more of the target whenever the target speaks, however the extension is able to remove more of the interfering speaker's energy (SIR: extension 10.09, WeSep 8.40 dB), keep the other speaker out when the target is absent (words per trial: extension 12.88, WeSep 17.35) and run in real time at half of WeSep's delay. All results are for at most two-speaker mixtures and are optimised for Gemini.

Contents

Nomenclature **iv**

1. Introduction **1**

2. Literature review **2**

2.1. Target speaker extraction . . . 2
2.2. Streaming separation . . . 2
2.3. How extraction is evaluated . . . 3
2.4. Speech LLMs as listeners and judges . . . 4

3. Methodology **5**

3.1. Data construction . . . 5
3.1.1. Why the data is constructed . . . 5
3.1.2. Trial construction . . . 6
3.1.3. Real-recording evaluation set . . . 8
3.2. Baseline model architecture . . . 8
3.2.1. Band-split Recurrent Neural Network (BSRNN) . . . 8
3.2.2. Short-time Fourier transform . . . 8
3.2.3. Band plan . . . 8
3.2.4. Per-band normalisation . . . 9
3.2.5. Residual LSTM block . . . 9
3.2.6. Band and sequence model . . . 9
3.2.7. Estimation module . . . 10
3.2.8. Time-frequency map . . . 10
3.2.9. Objective function . . . 11
3.3. Extension model architecture . . . 14
3.3.1. Splitting the cue into parts . . . 14
3.3.2. Speaker context embedding . . . 15
3.3.3. Target presence gate . . . 15
3.4. Metrics . . . 20
3.4.1. Transcripts and references . . . 20
3.4.2. Listener Content Fidelity Word Error Rate . . . 20
3.4.3. Interferer Content Rate . . . 20
3.4.4. Fabrication Rate . . . 21
3.4.5. Deep Noise Suppression Mean Opinion Score . . . 22
3.4.6. Signal-to-Distortion/Interference/Artefact Ratios . . . 22
3.4.7. Reference points . . . 22
3.4.8. Uncertainty . . . 22

4. Experiments **23**
4.1. Training setup . . . 23
4.1.1. Data augmentation . . . 23
4.1.2. Optimisation and hardware . . . 23
4.1.3. Model selection . . . 23
4.1.4. Training curves . . . 24
4.1.5. Systems compared . . . 26
4.2. Evaluation setup . . . 26
4.2.1. Listeners . . . 26
4.2.2. Latency measures . . . 27

5. Results **28**
5.1. RQ1: Conventional measures and the listener . . . 28
5.2. RQ2: Leaked and invented words . . . 29
5.3. RQ3: Voice information in real time . . . 32
5.4. Unseen and real recordings . . . 33

6. Conclusion **34**

Plagiarism and AI declaration **36**

References **38**

A. Construction parameters **43**

B. AMI evaluation set **44**

C. Presence gate **45**
C.1. Markov prior . . . 45

D. Evaluation protocol details **45**
D.1. Prompt for live-model transcription . . . 45
D.2. Pinned components of the measuring instrument . . . 46

Nomenclature

Abbreviations

AMI	Augmented multi-party interaction
API	Application programming interface
ASR	Automatic speech recognition
BAK	Background intrusiveness (DNSMOS P.835 sub-score)
BSRNN	Band-split recurrent neural network
CARTSE	Li and Seki's online REAL-TSE system [1]
CPU	Central processing unit
DNSMOS	Deep noise suppression mean opinion score
ECAPA-TDNN	Emphasised channel attention, propagation and aggregation TDNN
ELBO	Evidence lower bound
EQ	Equalisation
FR	Fabrication rate
ICR	Interferer content rate
ITU-T	ITU Telecommunication Standardization Sector
KL	Kullback–Leibler (divergence)
LAM	Large audio model
LCF-WER	Listener content fidelity word error rate
LLM	Large language model
LSTM	Long short-term memory
LUFS	Loudness units relative to full scale
MLP	Multilayer perceptron
NLTK	Natural Language Toolkit
OVRL	Overall quality (DNSMOS P.835 sub-score)
REAL-TSE	Real-world target speaker extraction (challenge)
RMS	Root mean square
RNN	Recurrent neural network
RTF	Real-time factor
SAR	Signal-to-artefacts ratio
SDR	Signal-to-distortion ratio
SIG	Speech signal quality (DNSMOS P.835 sub-score)
SIR	Signal-to-interference ratio
SI-SDR	Scale-invariant signal-to-distortion ratio
SNR	Signal-to-noise ratio
STFT	Short-time Fourier transform
TDNN	Time-delay neural network

TF-Map	Time-frequency map
TSE	Target speaker extraction
VAD	Voice activity detection
WER	Word error rate

Symbols

Signals and transcriptions

x	Mixture signal
s	Target reference signal
$\hat{s}$	Model output, the estimated target
s_{int}	Interfering speech signal

Model and objective

X, X_t	STFT of the mixture; frame t of it
X_e	STFT of the enrollment
S, $\hat{S}$	STFT of the target; of the model output
$\Re(\cdot)$, $\Im(\cdot)$	Real and imaginary parts of a complex value
N	STFT window length
$\tilde{x}_t$, $\tilde{e}_j$	Unit-length mixture frame; unit-length enrollment frame
$C_{t,j}$	Cosine similarity of mixture frame t and enrollment frame j
$A_{t,j}$	Softmax weight of enrollment frame j for mixture frame t
μ_t, u_t	Blended enrollment template; its unit-length direction
m_t, ϕ_t	Projection of $\lvert X_t\rvert$ onto u_t; the baseline cue $m_t u_t$
c_t, r_t	Similarity of $\lvert X_t\rvert$ to u_t; residual $\lvert X_t\rvert - m_t u_t$
M, R	Complex mask; residual spectrogram
α, s_{proj}	Gain of the output along the target; αs
e_{int}, e_{art}	Interference (other speaker and noise); artefact
L_{pres}, L_{MR}, L_{gain}, L_{abs}	Loss terms
w, w_{m}, w_{g}	Loss weights
$\varepsilon_{\text{pres}}$, ε_{abs}	Floors of L_{pres} (10^{-3}) and L_{abs} (10^{-2})

AI usage citations

To save space for explanations of how I used AI tools in specific parts of the project, I have made the specific declarations in the section which I reference using tags such as (AI1) and (AI7b).

1. Introduction

Voice assistants engage with users through direct, live speech-to-speech models such as Gemini Live [2]. With the appearance of more voice-based applications, the ability to engage with these tools in acoustically challenging environments is becoming increasingly important. A particular problem is the presence of interfering speech, which a speech model misinterprets as the intended target user speech, and thus responds to words never said by the target. Secondly, live speech models are exposed to noisy environments that include reverberation, background audio and other acoustic challenges. Although live models have been found robust to added white noise [3] (Section 2.4), processing audio to remove interfering speech can introduce artefacts that speech recognisers are not robust to [4, 5]. The work in this report addresses these issues through the development of a target speaker extractor [6] that aims to preprocess the audio input that is given to the live model. The extractor is constructed under the following constraints: it must be causal, process audio in real time and operate on a single channel of audio.

In this report the *listener* is the speech model that transcribes the extractor's output. The deployment target is Gemini Live [2] though every measurement uses Gemini 3.7 Flash [7] as its stand-in and is used to select checkpointed models. This work asks three questions:

RQ1 Do the measures conventionally used to evaluate target speaker extraction (signal-to-distortion ratios and predicted opinion scores) indicate how much of the target's speech the listener recovers from the extractor's output? (Section 5.1)

RQ2 When an extractor suppresses the interfering speaker, what are the listener's remaining errors and how are the errors divided between words leaked from the interfering speaker and words that neither speaker said? Does reducing one type of error increase the other? (Section 5.2)

RQ3 Within a causal extractor that runs in real time, does giving the network more explicit information about the target's voice reduce the listener's errors and does it improve the model's ability to interpret the target? (Section 5.3)

An evaluation suite in which the listener's errors are categorised into leaked versus invented words is developed. To answer the three research questions, this work builds a constructed dataset of 9955 training trials from 1172 LibriSpeech [8] speakers with exact transcriptions and target-absent trials. A trial consists of at most two speakers, noise and reverberation. A held-out test set of 248 trials from 20 speakers is used for final evaluation. Secondly, a causal implementation of the REAL-TSE model [9] with a proxy objective adapted from CARTSE [1] is developed with different methods to enhance speaker identity. Finally to test on real-world data, a set of AMI meeting recordings [10] serves as a second test set.

The remainder of this report is structured as follows: Section 2 reviews the background information and context for the target speaker extraction domain. Section 3 describes the

process of generating the data, the architecture of the extractor and the metrics. Section 4 sets out how the model is trained, selected and evaluated. Section 5 presents the results of the evaluation and Section 6 concludes the report with a summary and potential future work.

2. Literature review

The section presents and reviews target speaker extraction, streaming separation, how extraction is typically measured, and speech models as listeners (AI1, AI2) and is structured to present the gaps of the research questions.

2.1. Target speaker extraction

Target speaker extraction (TSE) is a well-established task in speech processing. Meeting transcription systems, hearing aids, and smart speakers all benefit from the ability for more robust speech recognition in the presence of background noise and competing speakers. The main idea is best described by Cherry's cocktail party problem [11], which is a description of a human's ability to focus on a single speaker in noisy environments. Using various indicators such as visual, spatial, and acoustic cues, a listener can understand the speech of a target speaker while ignoring other speakers. The survey done by Žmolíková et al. [6] provides a comprehensive overview, differentiating between three main tasks: target speaker extraction, blind source separation, and noise suppression. In this work, the focus is on target speaker extraction using only voice samples as an acoustic cue.

The survey by Žmolíková et al. [6] highlights the gap that many TSE systems are not trained to go silent when the target speaker is not present, which is a separate task to extraction. The confound of the target speaker always present during training leaves the model vulnerable to situations where there is only interfering speech or noise. Another issue raised by Žmolíková et al. [6] is that the use of signal-level metrics such as signal-to-distortion ratio (SDR) is indicative of the ability of the model to extract speech, but *(i)* it does not correlate with audible quality from the perspective of a human listener, and *(ii)* it does not reflect the ability of the model to select or identify a target speaker as an evaluation metric **(RQ1)**. The survey also notes that TSE systems are being moved to low-latency streaming setups. Therefore it is important to consider lightweight architectures that can be deployed in real-time scenarios **(RQ3)**. The first of the gaps is why the data in this work includes target-absent trials (Section 3.1.2).

2.2. Streaming separation

Separation models work with either a raw waveform or spectrogram. Some architectures such as Conv-TasNet [12] favoured the waveform with the reasoning that calculating a spectrogram contributes to the latency of the system. However, Yu et al. adapted a Band-Split RNN (BSRNN) architecture, originally designed for music as an offline model [13], to speech enhancement tasks with an online configuration [14]. Yu et al. found that their online implementation ran

faster than real time on an Intel i5 CPU.

TSE systems must work frame by frame, or chunk by chunk. The REAL-TSE challenge set a benchmark to say that a system must run with an algorithmic latency of $100\,\mathrm{ms}$ or less [9]. Algorithmic latency, separate from end-to-end latency, is the delay built into the model: how long it must wait for audio before it can produce an output. The assumption in this work is that a $200\,\mathrm{ms}$ to $300\,\mathrm{ms}$ end-to-end latency budget is sufficient. The architecture must also be causal. For the REAL-TSE baseline this meant replacing bidirectional LSTMs over time with unidirectional ones. It also included normalising over a single frame or chunk rather than over the whole sequence [9] **(RQ3)**.

A concern about streaming is that it makes it harder to identify the target speaker. As found in the survey by Žmolíková et al. [6], typical TSE systems use the entire context of an audio clip to identify the target speaker. However, in a streaming scenario, the system must identify the speaker as the audio arrives. It therefore remains open whether giving a causal extractor more explicit information about the target speaker's voice can make up for the loss of context, and still do so in a real-time budget **(RQ3)**.

2.3. How extraction is evaluated

When making use of signal-based metrics, practitioners are often required to use simulated data. This is because a clean reference signal is required for comparison against the model output. By nature it is quite difficult to obtain perfectly clean speech in a real-world scenario. Popular resources to use for simulated mixtures include Libri2Mix [15] and WSJ0-2mix [16]. Thus the first issue related to evaluation is the translation of performance on simulated data to real-world data. Zhang et al. [17] show that there is not necessarily a direct translation in performance: in multi-channel speech enhancement, the model with the best signal-to-distortion ratio (SDR) on simulated data more than doubled its word error rate (WER) compared with the unprocessed input on real recordings from 19.5 % to 41.5 %. Signal scores also fail in the following manner for TSE: *(i)* a low signal score does not say whether the extractor picked the wrong speaker or just separated the right one poorly, and *(ii)* a silent target leaves the score undefined **(RQ1)**.

Perceptual metrics such as DNSMOS [18] are designed to mimic what a human listener would perceive when listening to the audio. However, as found by Wang et al. [9], these metrics can be gamed by adding adversarial perturbations to the waveform to inflate them. In other words, an extractor can be trained to maximise the perceptual metric without it improving the actual quality of the audio. Secondly, these perceptual metrics still fail in TSE as they do not identify if the wrong person is speaking **(RQ1)**.

The next progression would be to transcribe the audio and compare it against a ground-truth transcription. The standard approach is to use the simple word error rate (WER) metric. However, this metric hides nuance in the failure of an extractor. Recognisers are hurt by the distortions of the extractor rather than by leftover noise in the audio [4] and secondly, a different recogniser may have different performance on the same audio **(RQ1)**. This leads into

the shortcomings of WER as a stand-alone metric for TSE. Suppose a word is heard from the interfering speaker, and secondly another word that neither speaker said and was invented, both can appear in the metric as a substitution or an insertion error **(RQ2)**. Finally the WER is only defined if there is target speech present.

2.4. Speech LLMs as listeners and judges

It is not a novel idea to make use of large language models (LLMs) to evaluate the quality of speech. In fact, Manakul et al. [3] found that large audio models (LAMs), which are LLMs that accept audio as input, are able to rank speech systems similarly to human raters. However, for tasks related to measuring speaking rate or speaker identity, the performance is poor and often close to random chance. The authors also found that the LAMs they tested were robust to added white noise, and more reliable on lexical content than on non-lexical content.

Using a language model as a rater is a studied problem. Zheng et al. [19] found that a strong text model, GPT-4, agreed with human preferences over 80 % of the time, the same level at which humans agree with each other. However, a caveat to this is that the model's verdicts carried biases: it preferred the first option it was shown, longer answers, and output that it had generated itself. The concern is that the same biases may be present in audio models. A partial solution came from AIR-Bench [20] that scores a pair of answers twice with the order reversed to prevent positional bias which is an idea carried through in the training of the model in this work where the target and interferer swap roles in an epoch to form two trials. Secondly, a model can leverage its prior knowledge of a language to make judgements instead of actually listening to the audio. This was explored by Foo et al. [21] where audio language models keep 60–72 % of their benchmark scores even when the audio is removed. The prior was found to influence hallucination too as found by Koenecke et al. [22] **(RQ2)**. Another consideration is the handling of noisy audio. A natural response would be to enhance the audio first. However, enhancement was found to be detrimental according to Chondhekar et al. [5] where the denoising of medical speech actually raised the semantic WER of all four recognisers (including Gemini Flash 2.0) that were tested in 40 different configurations **(RQ2)**.

In Manakul et al. the study uses the model as a rater. The LAM is prompted for a verdict, such as preference over audio samples, and the final decision becomes the score. This work uses the model differently. The live model is only asked to write down what it heard and the score is computed by comparing the transcription to a ground-truth text (Section 3.4.2). Each clip that is presented to the LLM is on its own, which mitigates positional bias that was found by Manakul et al. Their noise robustness was also only tested using white noise and did not explore competing speakers. Using the model as a listener rather than a rater is less common. The closest work reviewed here, Fela and Mowlaee [23], found that standard audio-quality metrics fail to predict how often speech enhancement changes an LLM's downstream output **(RQ1)**. Therefore a gap remains. The studies making use of a model as a listener consider noise instead of competing speakers, and report a single error instead of splitting it by source. None

separates words leaked from the interfering speaker from words that neither speaker said, or asks whether reducing one raises the other **(RQ2)**. Section 3.4.3 and Section 3.4.4 define two measures that make this distinction.

3. Methodology

The methodology section describes the process and reasoning behind the data generation, extractor architecture and evaluation metrics (Figure 3.1).

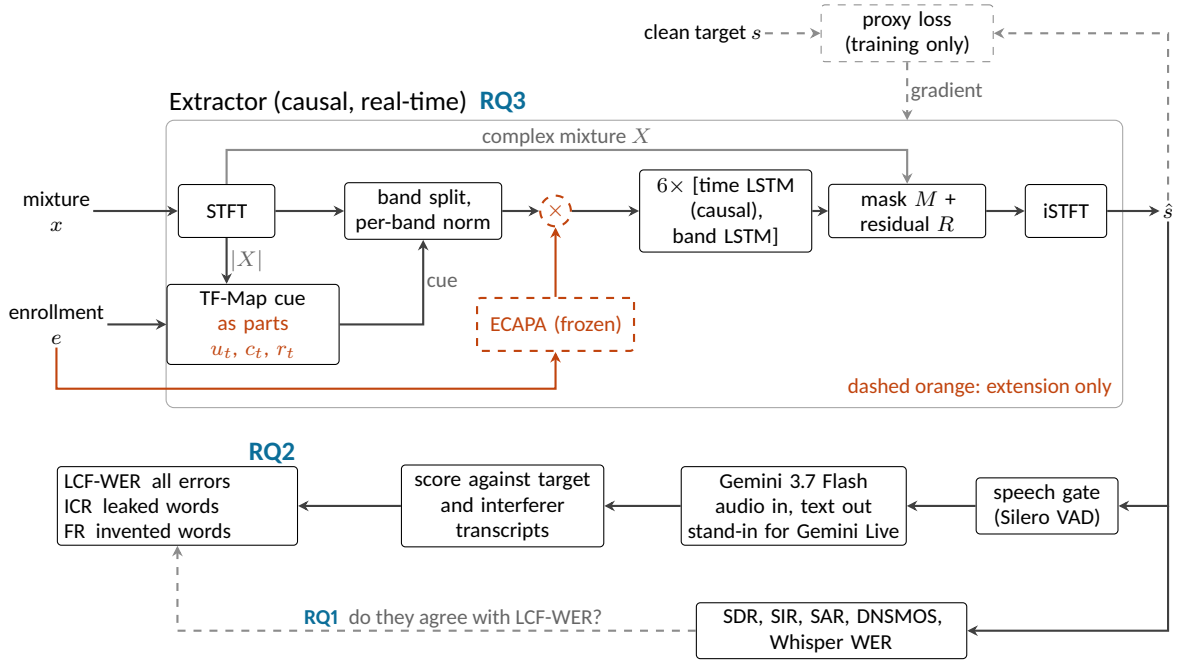


Figure 3.1: End-to-end pipeline. Top: the extractor. Orange blocks are the extension's additions: the cue passed as its parts (Section 3.3.1) and a frozen ECAPA embedding that scales the features (Section 3.3.2). Gemini cannot be differentiated through, so training uses a proxy loss against the clean target (Section 3.2.9). Bottom: evaluation. The output passes a speech gate, is transcribed by Gemini 3.7 Flash and is scored against both speakers' transcripts (Section 3.4). Bold tags mark where each research question is answered (AI3).

3.1. Data construction

3.1.1. Why the data is constructed

There are four main considerations for the dataset that need to be met: *(i)* training requires a clean target signal as a reference for what the extractor should output, *(ii)* the data must possess exact transcriptions of the target and interferer signals to allow for accurate transcription-based evaluation, *(iii)* trials need to include examples where the target is absent, and *(iv)* every trial has its unprocessed mixture and clean version to give the worst and best case for each trial. These considerations limit the available choices of publicly available datasets. The data thus serves all the research questions: a clean target makes the signal measures computable **(RQ1)**, exact transcriptions of both speakers make the leaked and invented word measures computable

(RQ2), and same-gender and low Signal-to-Interference Ratio (SIR) trials test whether the extractor picks the right voice **(RQ3)**.

Constructing the data is an established practice. The 2026 REAL-TSE challenge [9] supplied no training set at all, and required that participants document whichever open-source data they used. After reviewing the submissions, the organisers describe the data recipe that the entrants converged on: synthesise mixtures from LibriSpeech, WHAM! noise and simulated room impulse responses, whilst varying ratios such as the Signal-to-Noise Ratio (SNR), SIR, reverberation time, chunk length and enrollment quality and injecting target-absent segments. The dataset constructed here follows that recipe. The organisers further observed that several of the strongest entries were built on backbones near-identical to the official baseline architecture, the BSRNN, but attributed the success to data simulation, real-data adaptation, pseudo-label filtering, loss design and post-processing rather than to architecture alone.

3.1.2. Trial construction

The audio data was collected from a variety of sources to produce a diverse dataset. Speech was chosen for target and interferer from the LibriSpeech [8] dataset, which is a corpus of read English speech derived from audiobooks. The background noise was chosen from WHAM! [24], which is a dataset of real-world noise recordings considering environments such as offices, streets, and cafés. The trials aim to reflect real-world extraction scenarios. The primary focus is towards a base case of a target speaker close to a recording device with an interfering speaker in a similar proximity and faint background noise. The difficulty came in constructing the dataset to be realistic enough to simulate a real-world scenario, but also not so difficult that a simple extractor of limited capacity would be unable to learn the task. A visual representation of the trial construction can be found in Figure 3.2.

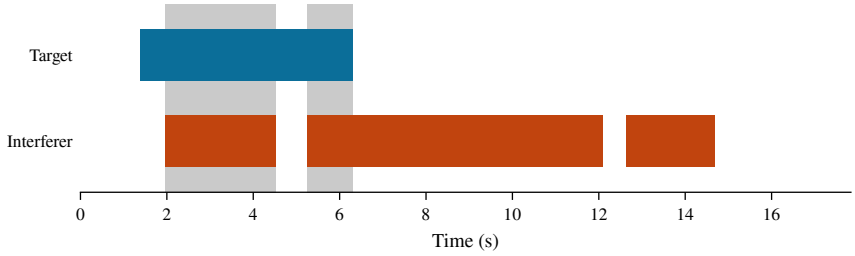


Figure 3.2: Visual representation of the trial construction. The target and interferer are drawn from LibriSpeech, and the noise bed is drawn from WHAM!. The shaded areas show the overlapping regions of speech.

A single trial consists of the following components: *(i)* target speech, *(ii)* interferer speech, *(iii)* noise bed, *(iv)* the mixture of the first three components, and *(v)* an enrollment clip of the target speaker. Each trial falls into one of four cases: *Case 1*, both speakers present, *Case 2*, the target alone, *Case 3*, the other speaker alone, and *Case 4*, noise only. The mixture is constructed to be 15–20 seconds long. The trials are drawn from two separate regimes: a base case and a difficult case. Base trials (60 %) use a higher SNR, a shorter T_{60} and more target speech, and difficult trials (40 %) use the full range of every parameter. The SIR range is the same

in both. Half of the two-speaker trials pair speakers of the same gender. To simulate room acoustics, including reverberation, room dimensions, source and microphone positions, the `pyroomacoustics` [25] package is used to simulate the room impulse response. The trials are constructed to be as realistic as possible, but also to be reproducible and controllable. Finally the noise clips that are too short are looped.

Enrollment. The enrollment clip is a clean $5\,\mathrm{s}$ audio segment of the target speaker from LibriSpeech and is used to provide a reference of what the target speaker sounds like. The enrollment clip is first attempted to be drawn from a different book reading, otherwise the clip is drawn from a different chapter of the same book, otherwise the clip is drawn from a different utterance of the same chapter. The reason for this is to avoid the extractor memorising the target speech by learning to recognise the content rather than the *manner* that the target speaker speaks. Again to avoid bias, the enrollment clips are simulated to come from different recording devices. This is done by applying equalisation (EQ) augmentation to the enrollment clips by applying a random EQ filter to the clip with a probability of 0.5. This technique was adopted from the CARTSE [1] submission for the 2026 REAL-TSE challenge [9].

Parameter distribution choice. SIR is chosen uniformly from the range of -10 to 10 dB, and the SNR is chosen uniformly from the range of 0 to 20 dB. These two ratios simulate the relative loudness of the target speaker to the interferer and background noise, respectively. A negative number for the SIR indicates that the interferer is louder than the target speaker, and the lower the SNR, the louder the background noise is relative to the target speaker. The SIR was chosen to be symmetric around $0\,\mathrm{dB}$ after experimentation found that if the target was always (or the majority of the time) louder than the interferer, then the extractor would just learn to follow the speech that was loudest and hence ignore the conditioning of the enrollment audio.

Reverberation time, T_{60}, is drawn uniformly from $0.25\,\mathrm{s}$ to $0.6\,\mathrm{s}$, which covers a carpeted office to a large room with hard surfaces. The wall absorption is solved using the Sabine relation [26] rather than sampled and a room that cannot simulate the drawn reverberation time is redrawn. The remaining parameters are listed in Appendix A.

Trial cases. The extractor needs example trials to learn to rely on the conditioning of the enrollment audio to identify target speech. The target-absent trials support this idea by having the model learn to stay silent when the enrollment speaker is not present. The split is as follows for these trials: 49.5% both speakers present, 25.5% target only, 19.6% interferer only and 5.3% noise only.

Splits. The dataset is split into training, validation and test sets. Each of these sets has its own distinct set of speakers such that there is no overlap of speakers between the sets. The training set consists of 9955 trials drawn from 1172 unique speakers. The validation set holds 200 trials from 40 other speakers: 103 with both speakers present, 47 with the target alone, 42 with the other speaker alone and 8 with noise only. It is used during training to score every epoch and to choose the checkpoint (Section 4). A test set is kept apart as the final evaluation of the

simulated data seen in Section 5. It holds 499 trials from 20 further speakers: 248 with both speakers present, 115 with the target alone, 109 with the other speaker alone and 27 with noise only. No model was trained, tuned or selected on it, and each model is scored on it only after its checkpoint has been chosen.

3.1.3. Real-recording evaluation set

The constructed dataset is limited to simulated rooms and read speech. To provide a final evaluation of the extractor, the AMI corpus [10] is used. The AMI corpus contains 100 hours of real recordings of meetings with multiple speakers. Trials are kept to 10–20 seconds and are cut following the method of REAL-T [27]. The extractor is required to listen to a single table microphone and the enrollment audio is of the target speaker speaking alone into their headset from a different meeting.

A difficulty is that the AMI corpus does not have clean reference signals or verbatim transcriptions. Another difficulty is that the enrollment is from a headset which can pick up other speakers and background noise, so the enrollment becomes an approximation rather than a clean reference. Finally, the mixture is far quieter than in the constructed dataset: the median level of the target is $-48.6\,\mathrm{dB}$ on AMI, compared with $-29.2\,\mathrm{dB}$ on the validation set. The AMI evaluation contains 300 trials from 58 meetings, totalling 77.9 minutes. There are 60 speakers and all trials are Case 1 (both speakers present). The full summary can be found in Appendix B.

3.2. Baseline model architecture

The baseline architecture section describes the architecture that was used as a backbone in the REAL-TSE baseline [9]. The architecture in this work is built from scratch, but the design is that of the baseline.

3.2.1. Band-split Recurrent Neural Network (BSRNN)

Luo and Yu [13] propose the BSRNN model as a frequency-domain separation model that is designed for high-sample-rate audio. The implemented BSRNN is adapted to speech with a TF-Map enrollment conditioning module [28] to turn it into a target speaker extraction model.

3.2.2. Short-time Fourier transform

The model operates in the time-frequency domain rather than on the waveform. To ensure the input audio is in suitable form, a short-time Fourier transform (STFT) is applied by cutting the waveform into short, overlapping frames. A window function is applied to each frame, and then a discrete Fourier transform is applied to each windowed frame. The result is a complex-valued spectrogram, a two-dimensional representation of the audio signal. On the horizontal axis is time, and on the vertical axis is frequency.

3.2.3. Band plan

The band plan in this work follows the implementation from Zhang et al. [28] for their $16\,\mathrm{kHz}$ target speaker extraction model, which is also used by the challenge baseline built on it. The

split is 257 frequency bins into 32 bands, but no reason is given in the original paper for these particular boundaries, and no explanation is given in previous papers. More bands are allocated to the lower frequencies as speech is more densely concentrated there and requires more resolution.

3.2.4. Per-band normalisation

Normalisation is applied across the values of a band *within a single frame*. This is a deliberate design choice that *opposes* Yu et al. [14], who use batch normalisation for their online configuration. The opposition is for the following reasons: *(i)* during training, the mean and variance are computed from the current batch, but during inference, these statistics are computed from the entire training set. The function that is optimised during training is therefore not the function that is deployed. *(ii)* A batch of three trials is too small to estimate the statistics reliably. *(iii)* Single-frame normalisation carries no state and is unaffected by the amount of audio processed, so a four-second training chunk is normalised exactly as a stream that has been running for a minute.

3.2.5. Residual LSTM block

Two kinds of sequential structures are modelled: time and frequency. Time is unidirectional, but frequency is bidirectional, as all of the bands of a frame are available at the same instant and can be traversed in either direction. The block is used for both kinds of modelling, with the same structure but different parameters. At inference time, since the model is running in a streaming fashion, the LSTM's internal state is preserved between frames and information can be carried forward indefinitely. Following Luo and Yu [13], each block works as follows: the input is normalised, passed through the LSTM, and projected back to the original width. Finally, the input is added back to the output of the projection, forming a residual connection. A single LSTM layer is given a width of 192, which is Yu et al.'s [14] choice for their $16\,\mathrm{kHz}$ model. The input to the block is 128 wide, which is projected to 192 for the LSTM and then projected back to 128 for the residual addition.

3.2.6. Band and sequence model

The band and sequence model is the implementation of the residual LSTM block of Section 3.2.5 in a stack of six blocks again following Yu et al. [14], using the input from Section 3.2.4. After the per-band normalisation and projection, there are 32 bands of 128 features each. Each block makes two passes: over time (unidirectional), then over frequency (bidirectional). Pairing the two passes in a single block is the design of Luo and Yu [13], who motivate it by analogy with dual-path recurrent networks for speech separation rather than by an argument specific to bands. The effect of stacking the blocks is to allow for the accumulation of evidence over longer time spans.

3.2.7. Estimation module

The estimation module is responsible for producing an output spectrogram that contains the estimated target speaker's speech. This is done by creating an output network per band to reconstruct the target speaker's speech from the features produced by the band and sequence model. Yu et al. realised it would be more efficient to mask the mixture rather than predict the output spectrogram directly. This mask is a number for each time-frequency bin, which the mixture is multiplied by. A mask alone is not enough, as multiplication can only ever scale what a bin already holds. In the cases where the interferer completely dominates a bin, no multiplier recovers the target, as the multiplication operation would not be able to scale the drowned-out target. To solve this, the estimation module makes use of a residual spectrogram, which is predicted directly and added. Yu et al. [14] introduce it for exactly this reason, to absorb what the mask alone produces as artefacts. Leaving this mask unbounded is useful, and is what the original implementation encouraged. The estimate for a band is then

$$\hat{S} = M \otimes X + R, \tag{3.1}$$

where $\otimes$ is the complex product, taken bin by bin. The X that the mask multiplies is the original complex mixture and is taken as is from the band split and not from the output of the model's features. The features are used only to *decide* how to mask the input. The conditioning feature of Section 3.2.8 therefore never enters the signal path either.

3.2.8. Time-frequency map

The TF-Map [28] is the module that turns speech enhancement into target speaker extraction. It compares the mixture with the clean enrollment clip frame by frame, and gives the network a cue of what the target's spectrum should look like at each instant. Let $|X_t|$ be frame t of the mixture's magnitude spectrogram and $|X_{e,j}|$ frame j of the enrollment's. Every frame is first scaled to unit length, so that only the spectral *shape* remains, not the loudness:

$$\tilde{x}_t = \frac{|X_t|}{\big\||X_t|\big\|}, \qquad \tilde{e}_j = \frac{|X_{e,j}|}{\big\||X_{e,j}|\big\|}. \tag{3.2}$$

Each mixture frame is then compared with every enrollment frame by cosine similarity. For unit vectors this is simply the inner product. The similarities are then turned into weights that blend the enrollment frames into a template that can be referenced against the mixture frame:

$$C_{t,j} = \big\langle \tilde{x}_t,\ \tilde{e}_j \big\rangle, \qquad A_{t,j} = \operatorname*{softmax}_j \big(\kappa\, C_{t,j}\big), \qquad \mu_t = \sum_{j=1}^{T_e} A_{t,j}\, \tilde{e}_j. \tag{3.3}$$

Zhang et al. [28] also normalise the frames and make use of cosine similarity, but they use softmax directly on the inner product rather than scaling it by a factor $\kappa = 16$. After testing it was found that since the similarities lie in $[0, 1]$, the softmax spread the weights too evenly

and thus did not give a clear template. The κ factor is therefore used to scale the similarities, which sharpens the softmax to give a more distinct template. The template only describes the shape and does not indicate whether a target is speaking. As in [28], the energy is restored by projecting the mixture frame onto the template's direction:

$$u_t = \frac{\mu_t}{\|\mu_t\|}, \qquad m_t = \Big\langle\, |X_t|,\; u_t \,\Big\rangle, \qquad \phi_t = m_t\, u_t, \tag{3.4}$$

where m_t is a scalar per frame which describes how much of the mixture's energy lies along the target's predicted spectrum. ϕ_t, the template scaled by it, is the cue passed to the network. The TF-Map itself has nothing to train as (3.2)–(3.4) contain no weights. The extra channel is the only addition as it widens the per-band projection by about $33\,000$ parameters.

3.2.9. Objective function

The objective has to express two different requirements: output the target signal when it is present, and output silence otherwise. A single loss cannot serve both use cases and so the objective is split, and each crop is scored by the branch (target present versus absent) that applies to it.

Throughout, x is the mixture, s the target reference and $\hat{s}$ the model's output, all in the time domain over one training crop.

Target present. The first term measures how much of the output is the target and how much is everything else, following [1]. The loss needs to be invariant to the overall volume: if $\hat{s}$ has the correct shape, it should not be penalised for being loud or quiet. Therefore the first step is to find the single gain that best explains the output as a scaled copy of the target, which will be calculated as a projection,

$$\alpha = \frac{\langle \hat{s},\, s \rangle}{\|s\|^2}, \qquad s_{\text{proj}} = \alpha s, \tag{3.5}$$

and then to compare the energy of that scaled target against the energy of whatever the output contains beyond it. The loss is modelled after the Scale-Invariant Signal-to-Distortion Ratio (SI-SDR). The loss is therefore,

$$L_{\text{pres}} = -10 \log_{10} \frac{\|s_{\text{proj}}\|^2}{\|\hat{s} - s_{\text{proj}}\|^2 + \varepsilon_{\text{pres}} \|s_{\text{proj}}\|^2}. \tag{3.6}$$

Lower is better. The $\varepsilon_{\text{pres}} = 10^{-3}$ term in the denominator is a floor and caps the best score at $-30\,\text{dB}$, which is a suggestion from CARTSE [1]. An output whose residual error already sits $30\,\text{dB}$ below the target is close enough to be treated as solved, so it would be wasteful to keep spending gradient on it. The measure has two blind spots. It views the leaked interferer, noise and artefacts as an identical error per unit of energy. Secondly, the ratio is dominated by the low frequencies and thus the quieter bands that carry the consonants can be reconstructed poorly without much penalty.

Multi-resolution spectral term. Equation (3.6) cannot express *where* in the spectrum the error lies: a model that reconstructs the loud low bands well and the quiet high bands poorly is barely penalised and, being scale-invariant, it does not constrain the output's absolute loudness at all. To address this, s and $\hat{s}$ are compared at several resolutions so that the reconstruction of both the low- and high-frequency bands at different scales is encouraged. A spectral term allows for this following [14]:

$$L_{\text{MR}} = \frac{1}{|\mathcal{N}|} \sum_{N \in \mathcal{N}} \Big(\big\| \, |S_N|^p - |\hat{S}_N|^p \, \big\|_1 + \big\| \Re S_N - \Re \hat{S}_N \big\|_1 + \big\| \Im S_N - \Im \hat{S}_N \big\|_1 \Big), \tag{3.7}$$

where S_N and $\hat{S}_N$ are the spectrograms of the target and the output (Section 3.2.2), with window length N and hop $N/4$, and each norm is averaged over its time-frequency coefficients.

The exponent $p = 0.3$ compresses magnitudes, which will highlight the quieter components relative to loud ones so that an error in a low-energy region is not diluted just because it is quiet. Without the compression the term would be dominated by the same loud low bands that already dominate (3.6). The real and imaginary terms are left uncompressed and they are what make the term sensitive to phase: a magnitude-only loss would be indifferent to the phase that the complex mask of Section 3.2.7 is free to change. Finally the term is evaluated at several window lengths namely: 8, 16, 32 and $64\,\text{ms}$. This is with respect to the model's own $32\,\text{ms}$ framing. The model builds its output by masking $32\,\text{ms}$ frames and overlapping them, so the places where most errors would occur can be expected to exist at frame boundaries. The $8\,\text{ms}$ and $16\,\text{ms}$ windows resolve this and the $64\,\text{ms}$ window catches harmonic structure that the short ones blur.

Target absent. The model should penalise any output that is not silence. This is modelled after a modified version of the absolute level loss from CARTSE [1], where in this work the loss now contains the denominator term.

$$L_{\text{abs}} = 10 \log_{10} \frac{\|\hat{s}\|^2 + \varepsilon_{\text{abs}} \|x\|^2}{\|x\|^2}. \tag{3.8}$$

Including the denominator term makes the loss relative to the input, which is directly readable: $0\,\text{dB}$ means the output is as loud as the mixture, that is, the model did nothing; $-10\,\text{dB}$ means the interfering speech was suppressed by $10\,\text{dB}$. The same ε_{abs} floors the term at $-20\,\text{dB}$, beyond which the output counts as silent. A positive value means the model amplified the mixture, which is a direct failure. The inclusion of the denominator also allows for scale invariance, which puts the loss in the same units as the loss for the target-present case.

Output level. L_{pres} is scale-invariant and L_{abs} rewards silence, leaving a gap in the training that would otherwise allow a model to simply mute its output. This gap is filled by the third term, which has two jobs: *(i)* to prevent the indiscriminate output of silence, and *(ii)* to keep the output at a reasonable level. It bounds the output level against the target's, with a dead

zone of $\pm\delta$:

$$L_{\text{gain}} = \max\Big(0,\ \Big|20\log_{10}\frac{\mathrm{rms}(\hat{s})}{\mathrm{rms}(s)}\Big| - \delta\Big), \qquad \delta = 3\,\mathrm{dB}. \tag{3.9}$$

Inside $\pm 3\,\mathrm{dB}$ of the target's level the term is exactly zero, where rms is the root mean square. Its weight, $w_{\text{g}} = 1.69$, is derived rather than chosen: it is the geometric midpoint of the weight at which muting stops paying (where what muting saves on (3.8) equals what it costs on (3.9), both weighted as in (3.10)) and the weight where output level will outweigh the actual content extracted.

Combining the two branches. Each term is averaged over the crops it applies to, and the two averages are weighted against one another:

$$L = (1-w)\underset{\text{target present}}{\text{mean}}\Big[\,L_{\text{pres}} + w_{\text{m}}L_{\text{MR}} + w_{\text{g}}L_{\text{gain}}\,\Big] + w\underset{\text{target absent}}{\text{mean}}\Big[\,L_{\text{abs}}\,\Big]. \tag{3.10}$$

The decision to average over the crops within each branch rather than over the entire batch is deliberate. During an epoch, a random crop can draw a trial whose label says both target and interferer are present, but the crop itself may contain silence (occurs in $5.8\,\%$ of cases). The model should therefore be rewarded for silence where the loss should not be diluted by the presence of other crops in the batch. Therefore the branch a crop belongs to is decided from the audio of the cropped target itself, not from the trial's label.

Weights. The weight w on the silent branch is derived rather than chosen. Silent crops make up a measured $f_{\text{abs}} = 0.297$ of the training crops and, following [1], each silent crop is counted twice ($\eta = 2$). The present branch then carries $1 - f_{\text{abs}} = 0.703$ and the absent branch $\eta f_{\text{abs}} = 0.594$, and normalising the two to sum to one gives

$$w = \frac{\eta f_{\text{abs}}}{(1-f_{\text{abs}}) + \eta f_{\text{abs}}} = \frac{0.594}{1.297} = 0.458.$$

The weight w_{m} on the spectral term is derived from L_{pres} and L_{MR}. The reason for this term is that (3.6) and (3.7) are in different units. L_{pres} is in units with respect to a ratio in decibels, and L_{MR} is the mean absolute difference between compressed magnitudes. Therefore the weighting term w_{m} will reconcile the two terms to be on the same scale. This is done by evaluating on the output that does nothing at all, $\hat{s} = x$, over 300 training crops which will simulate the scenario for the average losses for an untrained model. The median values for L_{pres} and L_{MR} are -5.91 and 0.184 respectively. The aim is for the spectral term (multi-resolution) to be roughly $30\,\%$ of the present branch at the start of training. The calculation then becomes

$$w_{\text{m}} = \frac{0.3 \times |L_{\text{pres}}|}{L_{\text{MR}}} = \frac{0.3 \times 5.9088}{0.1842} = 9.62.$$

3.3. Extension model architecture

The extension architecture is based on the baseline architecture of Section 3.2, with three modifications. The first change is how the cue is passed to the network (Section 3.3.1). The second change gives the network more information about the target speaker's voice (Section 3.3.2). The final change adds a gate to better infer whether the target speaker is present in a frame (Section 3.3.3).

3.3.1. Splitting the cue into parts

The baseline architecture passes the network the TF-Map cue of [28] and includes the energy-recovery projection (3.4). By (3.2), every mixture frame is its loudness times its shape, $|X_t| = \left\||X_t|\right\| \tilde{x}_t$. Substituting this into m_t of (3.4) shows what the baseline's single number contains:

$$m_t = \left\langle\, |X_t|,\ u_t \,\right\rangle = \left\||X_t|\right\| \left\langle \tilde{x}_t,\ u_t \right\rangle = \left\||X_t|\right\| c_t, \tag{3.11}$$

where

$$c_t = \cos\theta_t = \left\langle \tilde{x}_t,\ u_t \right\rangle \tag{3.12}$$

is the cosine similarity of (3.3) which is measured with the blended template of u_t instead of an enrollment frame. The problem with m_t in the baseline architecture is that once the loudness and the similarity are multiplied into a scalar, the network cannot distinguish between a frame that is very loud but not similar to the target and a frame that is very similar but not loud. The idea is to pass the parts instead of the product (AI7a). Two parts are already defined: the template direction u_t of (3.4) and the similarity c_t of (3.12). The third part is new and is the residual

$$r_t = |X_t| - m_t u_t, \tag{3.13}$$

which is the part of the mixture frame that the template of the target does not explain. A property of the residual is that it is orthogonal to the template, shown by:

$$\left\langle r_t, u_t \right\rangle = \left\langle |X_t| - m_t u_t, u_t \right\rangle = \left\langle |X_t|, u_t \right\rangle - m_t \left\langle u_t, u_t \right\rangle = m_t - m_t = 0, \tag{3.14}$$

where the last equality uses the fact that u_t is a unit vector. The orthogonality thus allows a frame to be split exactly into the equation:

$$|X_t| = m_t u_t + r_t. \tag{3.15}$$

The baseline cue can be recovered exactly as:

$$\phi_t = c_t \left\||X_t|\right\| u_t. \tag{3.16}$$

The three parts replace the single-channel cue and thus the network input grows from 3 to 5 channels. The new parts do not carry new information, as each part can be derived from the baseline's inputs, but recovering the parts would require a square root and a division operation, which the network's 1×1 projections cannot compute.

3.3.2. Speaker context embedding

Even split into parts, the cue (Section 3.3.1) is described only by the enrollment's spectra. Target speakers that have similar pitch and vocal characteristics thus produce similar templates. Measured on the baseline architecture, the extractor was able to select the right voice $71.8\,\%$ of the time when two speakers were of different gender, but only $52.6\,\%$ when they were the same gender. The idea for this extension is to incorporate further target speaker identity. The second extension is to add a speaker embedding for each target speaker. Each enrollment clip is passed through a frozen speaker recognition network, ECAPA-TDNN [29] using the SpeechBrain checkpoint [30] trained on VoxCeleb1 and VoxCeleb2 [31, 32], to produce a 192-dimensional embedding. The embedding is then passed through a small learned layer that maps a 192-dimensional embedding of e to a 128-dimensional vector to be used in the extractor. ECAPA itself is large at 20.77 million parameters and hence would cause storage concerns if it were included in the extractor. However, the ECAPA network is run once per enrollment clip and the resulting embedding is stored, so ECAPA never runs while streaming.

The WeSep toolkit [33], an established open-source implementation of the baseline architecture, supports two uses of the embedding. The first (which is the one used in this work) is to use an utterance-level embedding, and the second is to use the contextual embedding of Zhang et al. [28]. Although WeSep supports the contextual embedding, it was discarded as an option for this work because it recomputes the embedding for every frame of the mixture, which is computationally expensive.

3.3.3. Target presence gate

The idea of the gate is to target leaked words **(RQ2)**. The gate should silence the frames in which the target speaker is not present. The baseline architecture has no explicit, underlying mechanism to determine whether the target speaker is present in a frame. The gate provides this probabilistic estimate and is trained jointly with the extractor. The gate represents a two-state Markov chain, where the states are *present* and *absent*.

Gated output. Let $Z_t \in \{0, 1\}$ be the random variable that is 1 if the target speaker is present in frame t and 0 otherwise. The extractor's output frame from (3.1) is then gated by z_t, which indicates whether the target speaker is present:

$$\hat{S}_t(z_t) = z_t\Big(M_t \otimes X_t + R_t\Big). \tag{3.17}$$

The gate reads the separator output $y_{k,t} \in \mathbb{R}^{128}$ of band $k \in \{1, \dots, 32\}$ (Section 3.2.6):

$$h_t = \frac{1}{32} \sum_{k=1}^{32} y_{k,t}, \qquad \hat{h}_{t,n} = \frac{h_{t,n} - \bar{h}_t}{\sqrt{\mathrm{var}(h_t) + \epsilon}}, \qquad \pi_t = \sigma(\omega^\top \hat{h}_t + \omega_0), \tag{3.18}$$

with $\bar{h}_t = \frac{1}{128} \sum_{n=1}^{128} h_{t,n}$, $\mathrm{var}(h_t) = \frac{1}{128} \sum_{n=1}^{128} (h_{t,n} - \bar{h}_t)^2$ and $\epsilon = 10^{-5}$. Here h_t is the average of the band states, $\hat{h}_t$ is that state standardised across the channels. π_t is the probability that the target speaker is present in frame t, $\omega \in \mathbb{R}^{128}$, $\omega_0 \in \mathbb{R}$, and let $\psi = (\omega, \omega_0)$. The approximate posterior over the sequence of target presence variables $z_{1:T}$ [34] is then given by:

$$q_\psi(z_{1:T} \mid x, e) = \prod_{t=1}^{T} \mathrm{Bern}(z_t; \pi_t), \tag{3.19}$$

where x is the mixture and e is the enrollment. The product over frames follows the mean-field assumption.

Markov prior. The prior leverages the first-order Markov property (as a target who speaks in one $8\,\mathrm{ms}$ frame is likely to speak in the next [35]) and is factorised as:

$$\begin{aligned} p(z_{1:T}) &= p(z_1) \prod_{t=2}^{T} p(z_t \mid z_{t-1}) \\ &= p(z_1) \prod_{t=2}^{T} A_{z_{t-1} z_t} \quad \text{where } A_{ij} = p(z_t{=}j \mid z_{t-1}{=}i). \end{aligned} \tag{3.20}$$

A is fixed from the training labels and is the transition matrix of the Markov chain. Let D denote the random variable for the number of consecutive frames of a speech run (consecutive $z_t = 1$). Therefore, expressed as a probability:

$$\begin{aligned} p(D = d) &= A_{11}^{d-1} A_{10}, \qquad d \in \{1, 2, \dots\} \\ &= A_{11}^{d-1} (1 - A_{11}), \end{aligned} \tag{3.21}$$

therefore D follows a geometric distribution with

$$\mathbb{E}[D] = \frac{1}{1 - A_{11}}. \tag{3.22}$$

Next, to set the transition matrix A, the sample mean speech-run length in the training data, $\bar{d}_{\text{on}}$, is the maximum likelihood estimate of $\mathbb{E}[D]$, and treating silence runs (sample mean $\bar{d}_{\text{off}}$) the same way gives

$$A_{11} = 1 - \frac{1}{\bar{d}_{\text{on}}}, \quad A_{10} = \frac{1}{\bar{d}_{\text{on}}}, \quad A_{00} = 1 - \frac{1}{\bar{d}_{\text{off}}}, \quad A_{01} = \frac{1}{\bar{d}_{\text{off}}}. \tag{3.23}$$

Frame 1 has no previous frame, so it takes the stationary probability ρ of the chain. By the law of total probability, with $P(z_t = 1) = P(z_{t-1} = 1) = \rho$,

$$\rho = \rho A_{11} + (1 - \rho)A_{01} \implies \rho = \frac{A_{01}}{A_{01} + A_{10}}. \tag{3.24}$$

The actual transition matrix used in the prior can be found in Appendix C.

Objective. Training should make the clean target speech s as probable as possible given the mixture and the enrollment.

$$\begin{aligned}
\log p_\theta(s \mid x, e) &= \log \sum_z p_\theta(s \mid x, e, z)\, p(z) \\
&= \log \mathbb{E}_{z \sim q_\psi(z|x,e)} \left[\frac{p_\theta(s \mid x, e, z)\, p(z)}{q_\psi(z \mid x, e)} \right] \\
&\geq \mathbb{E}_{z \sim q_\psi(z|x,e)} \left[\log \frac{p_\theta(s \mid x, e, z)\, p(z)}{q_\psi(z \mid x, e)} \right] \quad \text{By Jensen's Inequality} \\
&= \mathbb{E}_{z \sim q_\psi(z|x,e)} \Big[\log p_\theta(s \mid x, e, z) \Big] - \mathbb{E}_{z \sim q_\psi(z|x,e)} \Big[\log q_\psi(z \mid x, e) - \log p(z) \Big] \\
&= \mathbb{E}_{z \sim q_\psi(z|x,e)} \Big[\log p_\theta(s \mid x, e, z) \Big] - \mathrm{KL}\Big(q_\psi(z \mid x, e) \,\|\, p(z) \Big).
\end{aligned} \tag{3.25}$$

The first term of (3.25) requires a probability, but only a loss is available during training: $L(\hat{s}, s)$, the loss of (3.10) computed on the gated output. Let the probability fall exponentially as the loss grows, with a temperature $\beta > 0$:

$$p_\theta(s \mid x, e, z) = \frac{1}{\mathcal{Z}} \exp\left(-\frac{L(\hat{s}(z), s)}{\beta} \right), \qquad \log p_\theta(s \mid x, e, z) = -\frac{L(\hat{s}(z), s)}{\beta} - \log \mathcal{Z}, \tag{3.26}$$

where $\mathcal{Z}$ is the normalisation constant. Substitute (3.26) into (3.25) to give:

$$\begin{aligned}
\log p_\theta(s \mid x, e) &\geq \mathbb{E}_{z \sim q_\psi(z|x,e)} \Big[-\frac{L(\hat{s}(z), s)}{\beta} - \log \mathcal{Z} \Big] - \mathrm{KL}\Big(q_\psi(z \mid x, e) \,\|\, p(z) \Big) \\
&= -\frac{1}{\beta} \mathbb{E}_{z \sim q_\psi(z|x,e)} \Big[L(\hat{s}(z), s) \Big] - \log \mathcal{Z} - \mathrm{KL}\Big(q_\psi(z \mid x, e) \,\|\, p(z) \Big).
\end{aligned} \tag{3.27}$$

Let the right-hand side of (3.27) form part of the objective to obtain

$$\begin{aligned}
\mathcal{L}(\theta, \psi) &= -\frac{1}{\beta}\, \mathbb{E}_{z \sim q_\psi(z|x,e)} \Big[L(\hat{s}(z), s) \Big] - \log \mathcal{Z} - \mathrm{KL}\Big(q_\psi(z \mid x, e) \,\|\, p(z) \Big) \\
-\beta\, \mathcal{L}(\theta, \psi) &= \mathbb{E}_{z \sim q_\psi(z|x,e)} \Big[L(\hat{s}(z), s) \Big] + \beta \log \mathcal{Z} + \beta\, \mathrm{KL}\Big(q_\psi(z \mid x, e) \,\|\, p(z) \Big) \\
&\propto \mathbb{E}_{z \sim q_\psi(z|x,e)} \Big[L(\hat{s}(z), s) \Big] + \beta\, \mathrm{KL}\Big(q_\psi(z \mid x, e) \,\|\, p(z) \Big).
\end{aligned} \tag{3.28}$$

The ELBO at this point does not use the labels v_t for whether the target is present and thus nothing forces π_t to follow whether the target is speaking. Since v_t is known for every frame in training, it is included in a binary cross-entropy between π_t and v_t. This idea was suggested by

Kingma et al. [36] in an image classification context. The final objective to train the presence gate is then:

$$\begin{aligned}\mathcal{J}(\theta,\psi) = \mathbb{E}_{z\sim q_\psi(z|x,e)}\Big[L(\hat{s}(z),s)\Big] + \beta\,\mathrm{KL}\Big(q_\psi(z\mid x,e)\,\|\,p(z)\Big)\\ - \lambda\sum_{t=1}^{T}\Big[v_t\log\pi_t + (1-v_t)\log(1-\pi_t)\Big],\end{aligned} \tag{3.29}$$

where λ weights the label term. The gated run used $\beta = 0.01274$ and $\lambda = 0.04689$.

Closed-form KL. The KL term of (3.28) under the mean-field posterior has a closed form. Suppose

$$\mathrm{KL}\Big(q_\psi(z\mid x,e)\,\|\,p(z)\Big) = \mathbb{E}_{z\sim q_\psi(z|x,e)}\Big[\log q_\psi(z\mid x,e)\Big] - \mathbb{E}_{z\sim q_\psi(z|x,e)}\Big[\log p(z)\Big], \tag{3.30}$$

then term 1 can be simplified as follows:

$$\begin{aligned}\log q_\psi(z\mid x,e) &= \sum_{t=1}^{T}\Big[z_t\log\pi_t + (1-z_t)\log(1-\pi_t)\Big]\\ \mathbb{E}_{z\sim q_\psi(z|x,e)}\Big[\log q_\psi(z\mid x,e)\Big] &= \sum_{t=1}^{T}\Big[\pi_t\log\pi_t + (1-\pi_t)\log(1-\pi_t)\Big],\end{aligned} \tag{3.31}$$

where $\mathbb{E}_{z\sim q_\psi(z|x,e)}\Big[z_t\Big] = \pi_t$. Term 2 simplifies as follows:

$$\log p(z) = \log p(z_1) + \sum_{t=2}^{T}\log A_{z_{t-1}z_t}. \tag{3.32}$$

Recall that being in state i at a time frame t is modelled by the Bernoulli distribution. Therefore, for the first frame only the case of belonging to a state with no transitions is considered, and from the second frame onwards, the transitions are considered. The expectation of (3.32) is then:

$$\begin{aligned}\mathbb{E}_{z\sim q_\psi(z|x,e)}\Big[\log p(z)\Big] &= \mathbb{E}_{z\sim q_\psi(z|x,e)}\Big[\log p(z_1) + \sum_{t=2}^{T}\log A_{z_{t-1}z_t}\Big]\\ &= \mathbb{E}_{z\sim q_\psi(z|x,e)}\Big[z_1\log\rho + (1-z_1)\log(1-\rho)\\ &\quad + \sum_{t=2}^{T}\Big(z_{t-1}z_t\log A_{11} + z_{t-1}(1-z_t)\log A_{10}\\ &\quad + (1-z_{t-1})z_t\log A_{01} + (1-z_{t-1})(1-z_t)\log A_{00}\Big)\Big]^{\dagger}\\ &= \pi_1\log\rho + (1-\pi_1)\log(1-\rho) +\\ &\quad \sum_{t=2}^{T}\Big[\pi_{t-1}\pi_t\log A_{11} + \pi_{t-1}(1-\pi_t)\log A_{10}\end{aligned}$$

$$+ (1-\pi_{t-1})\pi_t \log A_{01} + (1-\pi_{t-1})(1-\pi_t)\log A_{00}\Big]^{\ddagger}. \tag{3.33}$$

† where $p(z_1 = 1) = \rho$ from (3.24), and since $z_t \in \{0,1\}$ exactly one of the four products is 1 for each pair (z_{t-1}, z_t), which selects the matching transition. ‡ where $\mathbb{E}_z\Big[z_{t-1}z_t\Big] = \mathbb{E}_z\Big[z_{t-1}\Big]\mathbb{E}_z\Big[z_t\Big] = \pi_{t-1}\pi_t$ by the mean-field assumption, following the completely factorised variational inference of Ghahramani and Jordan [37]. Therefore subtracting (3.33) from (3.31) and removing $t = 1$ from the first summation yields

$$\begin{aligned}\mathrm{KL}\Big(q_\psi(z \mid x, e) \,\|\, p(z)\Big) = \pi_1\Big(\log\frac{\pi_1}{\rho}\Big) + (1-\pi_1)\Big(\log\frac{1-\pi_1}{1-\rho}\Big) + \\ \sum_{t=2}^{T}\Big[\pi_{t-1}\Big[\pi_t\log\frac{\pi_t}{A_{11}} + (1-\pi_t)\log\frac{1-\pi_t}{A_{10}}\Big] + \\ (1-\pi_{t-1})\Big[\pi_t\log\frac{\pi_t}{A_{01}} + (1-\pi_t)\log\frac{1-\pi_t}{A_{00}}\Big]\Big].\end{aligned} \tag{3.34}$$

Using the identity $\mathrm{KL}\Big(\mathrm{Bern}(a) \,\|\, \mathrm{Bern}(b)\Big) = a\log\frac{a}{b} + (1-a)\log\frac{1-a}{1-b}$, the KL term simplifies to:

$$\begin{aligned}\mathrm{KL}\Big(q_\psi(z \mid x, e) \,\|\, p(z)\Big) = \mathrm{KL}\Big(\mathrm{Bern}(\pi_1) \,\|\, \mathrm{Bern}(\rho)\Big) + \sum_{t=2}^{T}\Big[\pi_{t-1}\,\mathrm{KL}\Big(\mathrm{Bern}(\pi_t) \,\|\, \mathrm{Bern}(A_{11})\Big) + \\ (1-\pi_{t-1})\,\mathrm{KL}\Big(\mathrm{Bern}(\pi_t) \,\|\, \mathrm{Bern}(A_{01})\Big)\Big].\end{aligned} \tag{3.35}$$

Training and inference. The expectation of $L(\hat{s}(z), s)$ of (3.29) is too expensive to compute as an average over all 2^T possible sequences of $z_{1:T}$. Instead leverage Monte Carlo sampling to approximate the expectation (AI7b). Only one sequence of $z_{1:T}$ is drawn per training clip and is an unbiased estimate of the expectation [34]. A sample $z_t \in \{0,1\}$ drawn from a $\mathrm{Bern}(\pi_t)$, however, will potentially not change when π_t is changed slightly, so $\partial z_t/\partial\pi_t = 0$ and thus the extraction loss sends no gradient to the gate. This is solved using a soft draw with the binary Concrete distribution [38, 39].

$$\tilde{z}_t = \sigma\Big(\frac{1}{\tau}\Big(\log\frac{\pi_t}{1-\pi_t} + \log U_t - \log(1-U_t)\Big)\Big), \qquad U_t \sim \mathcal{U}(0,1), \tag{3.36}$$

where $\tau > 0$ is a temperature. The noise U_t does not depend on ψ, so $\tilde{z}_t$ is a differentiable function of π_t (the reparameterisation trick [34]), and as $\tau \to 0$ it tends to a hard $\mathrm{Bern}(\pi_t)$ draw. The temperature τ is annealed linearly per epoch from 1 to 0.5. At inference a single deterministic output is needed. The output that minimises the expected squared error to the random gated output (3.17) is its mean,

$$\hat{S}_t = \mathbb{E}_{z_t\sim\mathrm{Bern}(\pi_t)}\Big[z_t\,(M_t \otimes X_t + R_t)\Big] = \pi_t\,(M_t \otimes X_t + R_t), \tag{3.37}$$

so at test time each frame is scaled by π_t and nothing is sampled.

3.4. Metrics

The metrics are where the main contribution of the work lies. Since the main task is intelligibility of the target speaker for the listeners and not necessarily the *quality* of the extracted target speaker speech, the metrics that are designed in this section aim to reflect that objective. LCF-WER, ICR and FR measure what the listener recovers and split its errors into leaked and invented words, while DNSMOS and the signal-to-distortion ratios are the conventional measures compared against them.

3.4.1. Transcripts and references

When a system produces output, either the offline or the live model with the prompt of Appendix D.1, it gives a transcript. The two anchors get transcribed as well to give the untouched mixture and the clean target. Every transcript is scored against the LibriSpeech reference transcript of the target speaker, and the interferer speaker. Before scoring, each transcript is normalised using Whisper's `EnglishTextNormalizer` [40, 41]. This includes lowercasing, removing punctuation, expanding contractions and writing numbers consistently.

3.4.2. Listener Content Fidelity Word Error Rate

The Listener Content Fidelity Word Error Rate (LCF-WER) is a word error rate applied to a live-model transcription of the audio. The equation is as follows:

$$\text{WER} = \frac{S + D + I}{N} \tag{3.38}$$

where S is the number of substitutions, D is the number of deletions, I is the number of insertions, and N is the total number of words in the reference. It is essentially a minimum edit-distance formula, where the number of edits required to change a reference text into the proposed text is divided by the number of words in the reference text. The metric can go above 100 %.

3.4.3. Interferer Content Rate

The Interferer Content Rate (ICR) is the first metric used to highlight one particular nuance of the LCF-WER metric: when the live model transcribes a word that the target never said, that word was either invented by the model or heard from the interfering speaker. ICR measures the second case (AI8).

Content words. The ICR counts only content words. Function words such as *the*, *and* and *of* are removed with the NLTK English stopword list [42], because almost any two English sentences share them. An overlap on these words would say nothing about whether the other speaker's content reached the listener. These content words will be used to make an interferer-exclusive bag of words, which indicates which words the model picks up from the interfering speaker. Specifically, a content word is *available to leak* on trial i if the interfering speaker said it and the

target did not. Each word is counted once per trial: a_i is the number of words available to leak on trial i, and ℓ_i the number of those that leaked.

The rate. The number of leaked words grows with how much the interfering speaker said, so the count is turned into two rates. The first asks how often a trial leaks. For a set of trials $\mathcal{T}$ and a threshold of k words,

$$\text{ICR@}k = \frac{\left|\{\, i \in \mathcal{T}_k \;:\; \ell_i \geq k \,\}\right|}{|\mathcal{T}_k|}, \qquad \mathcal{T}_k = \{\, i \in \mathcal{T} \;:\; a_i \geq k \,\}, \tag{3.39}$$

where $\mathcal{T}$ is the set of trials being scored, i indexes a trial in it, ℓ_i is the number of leaked words on trial i, a_i is the number of words available to leak on trial i, k is the number of leaked words at which a trial counts as leaking, and $\mathcal{T}_k$ is the subset of trials with at least k words available to leak. Only trials with at least k available words are eligible, since a trial with a single available word can never reach ICR@2 and would only dilute the rate. This work uses $k = 2$: one shared content word can be coincidence, while two is a signal. The choice does not change any conclusion, as every k from 1 to 5 fully separates the no-processing and target-alone reference points (Section 3.4.7). The second rate asks how much leaks, as the share of the available words that reached the transcript, expressed as a percentage:

$$\text{mean leak} = \frac{1}{|\mathcal{T}|} \sum_{i \in \mathcal{T}} \frac{\ell_i}{a_i}. \tag{3.40}$$

Lower percentages are better for both metrics.

3.4.4. Fabrication Rate

The Fabrication Rate (FR) highlights the case where the model has invented a word that is present in neither the target nor the interferer and is only computable with exact transcriptions as given in LibriSpeech [8]. The FR metric is defined as follows: Over the set $\mathcal{T}$ of trials with a non-empty response, the statistic is the mean invented count per trial of the number of words in the model output that are not present in either the target or interferer,

$$\text{FR}_{\text{count}} = \frac{1}{|\mathcal{T}|} \sum_{i \in \mathcal{T}} \left|\mathcal{I}_i\right|, \tag{3.41}$$

where $\mathcal{I}_i$ is the set of words in the i-th trial's output that are not present in either the target or interferer (AI8). A rate of fabrication is also defined by whether a trial contains at least k invented words as defined in (3.42).

$$\text{FR@}k = \frac{1}{|\mathcal{T}|} \sum_{i \in \mathcal{T}} \mathbf{1}\left[\, \left|\mathcal{I}_i\right| \geq k \,\right], \qquad k = 2 \text{ chosen in this work}, \tag{3.42}$$

where $\mathbf{1}$is the indicator function. An invented word is not necessarily a hallucination as defined by [22]: a misheard word is counted as invented, but it is not a hallucination.

3.4.5. Deep Noise Suppression Mean Opinion Score

DNSMOS [18,43] is a neural network that predicts what a collection of humans would have rated an audio clip on a scale of 1–5, where 1 is bad and 5 is excellent. Two variants are reported: ITU-T P.835 [44], which gives speech quality (SIG), background quality (BAK) and overall quality (OVRL), and ITU-T P.808 [45], a single score that the REAL-TSE organisers adopted after entrants were found to game the P.835 metric.

3.4.6. Signal-to-Distortion/Interference/Artefact Ratios

SDR is used to measure how wrong the output audio is in total, and the SIR and SAR are used to measure in which direction the output audio is wrong. The SIR measures how much real audio that is not target speech is present in the output audio such as an interfering speaker or background noise, and the SAR measures how much of the output audio is not real audio, such as artefacts from the model. The output is split by orthogonal projection [46] into the target part s_{proj} of (3.5), the interference e_{int} explained by the other speaker s_{int} and the noise, and the artefact e_{art} explained by none of the sources, so that $\hat{s} = s_{\text{proj}} + e_{\text{int}} + e_{\text{art}}$:

$$\begin{aligned} \text{SDR} &= 10\log_{10}\frac{\|s_{\text{proj}}\|^2}{\|e_{\text{int}} + e_{\text{art}}\|^2 + \varepsilon_{\text{pres}}\|s_{\text{proj}}\|^2}, \\ \text{SIR} &= 10\log_{10}\frac{\|s_{\text{proj}}\|^2}{\|e_{\text{int}}\|^2 + \varepsilon_{\text{pres}}\|s_{\text{proj}}\|^2}, \\ \text{SAR} &= 10\log_{10}\frac{\|s_{\text{proj}} + e_{\text{int}}\|^2}{\|e_{\text{art}}\|^2 + \varepsilon_{\text{pres}}\|s_{\text{proj}} + e_{\text{int}}\|^2}, \end{aligned} \tag{3.43}$$

which are all measured in decibels (dB) where the higher ratio means better extraction of the target from the other audio in question. Each denominator is floored at $\varepsilon_{\text{pres}} = 10^{-3}$ of its numerator, as in (3.6), which caps every ratio at $+30\,\text{dB}$.

3.4.7. Reference points

The following terms are used to describe the lower and upper bounds of the results section. *No processing* refers to passing the full mixture audio, unprocessed, to a listener as the worst-case scenario, and *target alone* refers to passing the clean target audio to the listener as the best case. Target alone is perfect separation, where the listener hears only the target audio. Neither is a real system; both are reference points to compare the extractors against.

3.4.8. Uncertainty

Each 95 % interval is a percentile bootstrap [47]. The interval is computed using bootstrap resampling where the trials are redrawn with replacement 10,000 times, and on every draw the score is recomputed as the total errors over the total reference words [48]. Because trials from the same speaker are not independent, unique speakers are chosen rather than individual trials, and whole meetings for the AMI recordings.

4. Experiments

This section discusses how the models are trained and selected, and how they are evaluated. Firstly, the conventional measures and the Gemini-based metrics are computed on the same outputs to see whether they agree **(RQ1)**. Secondly, fine-tuning runs are trained where each run weights one kind of error more heavily, to demonstrate how the leaked and invented errors interact **(RQ2)**. Finally, the baseline and the extension are compared on Gemini's errors, how often the right voice is picked and the delay, to show what the voice information adds in real time **(RQ3)**.

4.1. Training setup

The training setup is described below and considers how the training data is augmented, the optimisation and hardware used, how the final model is selected, the systems compared, and the actual training observed.

4.1.1. Data augmentation

The training data is augmented with varying strategies to prevent overfitting. The first strategy is to provide every trial twice per epoch where the target and interferer roles are swapped. This means the same mixture requires two different outputs where the only varying information is the enrollment clip. If a clip is only used once for a single speaker, the answer remains the same which can be memorised. Therefore the swap ensures that if the model memorises one voice, it will be penalised if it fails to generalise to the other voice. The second strategy is to rotate enrollment clips for a target speaker. Each speaker per trial rotates between 3 enrollment clips so that the model does not memorise a single enrollment to a speaker. The third strategy is to randomly perturb the noise and reverberation for each trial per epoch. Finally, in the spirit of stochastic methods, a single random $4\,\mathrm{s}$ crop is taken from a random position in the trial for each epoch.

4.1.2. Optimisation and hardware

During training the AdamW optimiser [49] was used with a base learning rate of 0.0005, weight decay of 0.0001 and gradient clipping at 5.0. A batch size of 3 was used, thus 6 samples were processed per iteration (see Section 4.1.1) with mixed precision training. The learning rate schedule included halving the learning rate when the Whisper WER stops improving. The target-absent term in the loss, L_{abs}, is off for the first 4 epochs, as the model in the early epochs would just output silence as this term is easy to optimise. The model is trained on Kaggle's Tesla T4 GPU for 12 hours per session and the random seed is set to 42 for reproducibility (AI6).

4.1.3. Model selection

An early stopping mechanism after 10 epochs without improvement was available as a backstop but never triggered. The baseline ran for one session of 14 epochs. The extension ran for 14

epochs, and because its in-training word error rate was still falling at the end of that session (its best epoch was the last), training was resumed for a second session, to 28 epochs.

The final model to base the evaluation on cannot simply be chosen by the model with the lowest validation loss as a result of the proxy training objective. The loss (Section 3.2.9) is a proxy for the downstream task, where the approach in the proxy is to reward better separation. However, better separation does not guarantee better recognition (Section 2.3): the epoch with the lowest loss for the extension has the highest LCF-WER of the chosen checkpoints as seen in Figure 4.1. Secondly, the model cannot be optimised directly against the LCF-WER as it is not differentiable. Finally a consideration is that Gemini is also too slow to run after every epoch. The model is thus chosen as follows: Whisper (Section 4.2.1) during training will create a shortlist of the best candidate checkpoints based on the WER of the validation set. The candidates are then scored by the live model (Section 4.2.1) on the validation set, where the lowest WER is chosen as the final model with ties broken by fewer invented words.

4.1.4. Training curves

Both models were trained with the same loss (except the additional presence gate in the extension) and the same settings, but differ in how long they were trained. The baseline was trained for 14 epochs and the extension for 28. Figure 4.1 shows how the validation scores of each model changed during training. The top row of Figure 4.1 shows the breakdown of the loss for the target-present branch. The three terms of the loss, shown separately, highlight how the terms interact with each other. A typical trend seen for both the baseline and the extension is that as the separation term L_{pres} for training decreases (rather sharply), the validation separation very gradually decreases, and then very gradually increases again. A positive note to be seen is how the multi-resolution spectrum term, L_{MR}, consistently decreases for both training and validation, showing how the extractor is able to better reconstruct the masked audio. The gain term for ensuring that the model produces audio at a similar level to the target, L_{gain}, follows the same trend as the separation term, which is a slight decrease followed by a slight increase. Finally, the baseline model is only trained for one session, as by the validation live model results it can be seen that on the final epoch the validation WER has returned to its epoch 4 score. Epoch 11 is chosen for the baseline as it has the lowest validation WER. Although separation is not the hallmark indicator of a good extractor for the context of the downstream task, when the term is decreasing in conjunction with the spectrum term, it is a good indication to continue training. The extension is thus trained for a second session as by the end of the first session, *(i)* the Whisper WER is generally decreasing although sporadic, and *(ii)* the separation term with the spectrum term is decreasing. Epochs 21 and 27 tie on validation LCF-WER (39.59 % against 39.23 %), but epoch 27 leaks fewer of the other speaker's words (20.16 % against 23.08 %) and invents more (2.56 against 2.20 words per trial). Epoch 21 is chosen because it invents fewer words. Finally, when considering the presence gate, it can be seen that the loss for the gate prior decreases steadily for both training and validation with an increase after epoch 25.

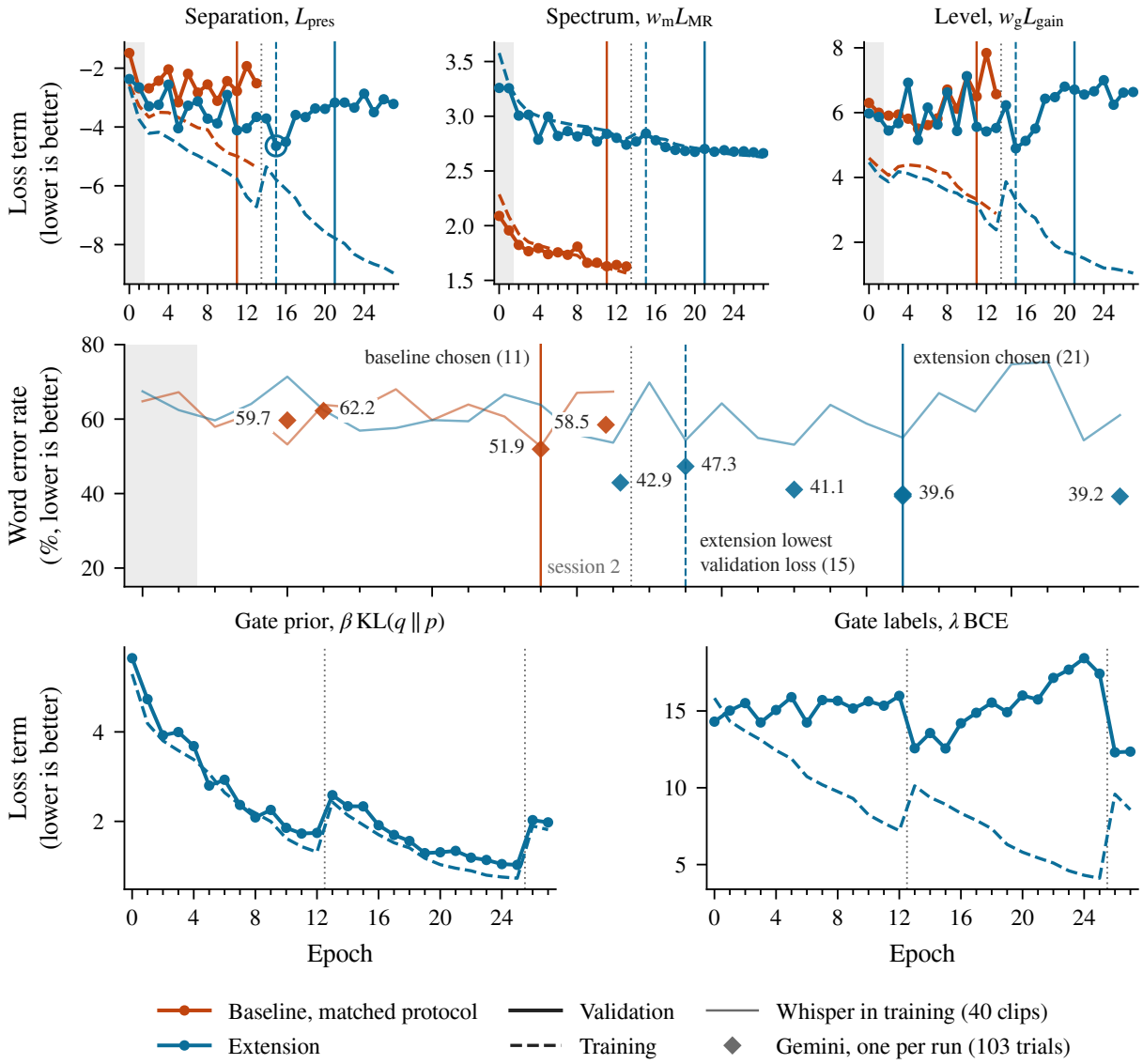


Figure 4.1: The baseline (orange) and the extension (blue), trained under the same protocol. The top row shows the three weighted terms of the target-present loss which are attributed to target-specific components, as seen in the first bracket of Equation 3.10. The middle row shows the word error rate on the 103 validation trials with both speakers present. Each diamond is one run of the live model on the same audio. The solid vertical line marks the chosen epochs, and the grey dotted line where training resumed in a second session. The bottom row shows the loss curve of the presence gate, which is only trained on the extension.

This shows that the model decision for the gate to turn off or on matches the prior's speech and silence durations. However, a concerning note is in the loss for the gate labels. The label loss decreases from 15 to 4 in training while validation does not indicate a clear downward trend, so the gate memorised when the training speakers talked. The gate is what allows speech to be passed through to the downstream model. It is clear that the model at the chosen epoch had not learnt anything in the training set that generalises to the validation set. Therefore, the gate is not used in the final model and the extension is evaluated without it.

4.1.5. Systems compared

This work evaluates three models against No processing and Target alone (Section 3.4.7): baseline BSRNN (implemented following REAL-TSE [9]; Section 3.2), the extension BSRNN (Section 3.3), and WeSep [33]. There are four main reasons to include WeSep for comparison: *(i)* the REAL-TSE challenge [9], which this work uses as a guideline, builds its baseline system with WeSep; *(ii)* the WeSep model has the same BSRNN backbone with the TF-Map cue of [28] as this work's baseline and extension, where the main architectural difference is WeSep's use of contextual embeddings rather than the single embedding per enrollment clip used by the extension, as discussed in Section 3.3.2; *(iii)* WeSep was trained on clean Libri2Mix [15] and has not heard any of the speakers in the evaluation sets, so it shows what an established model, trained on other data, recovers from these mixtures; and *(iv)* WeSep provides a stronger offline reference for extracting the target. The WeSep model itself is non-causal and almost four times the size of the baseline (27.2 M versus 7.19 M parameters), and thus shows what extraction can achieve when latency and model size are not a concern. WeSep was not trained on data that contained noise or reverberation, whereas trials in this work contain both. Secondly, WeSep itself has a causal separator; however, the features are normalised over the entire clip before the separator. This setup means that every output frame depends on audio that has not arrived yet and thus the model cannot be streamed and is treated as an offline system. Finally, the results of WeSep are not comparable to the results found in the REAL-TSE challenge [9] due to varying data, metrics and protocols.

4.2. Evaluation setup

The evaluation setup is described below and considers the listeners used, the latency measures, and the evaluation metrics. The evaluation metrics are described in Section 3.4 and the results of the evaluation are presented in Section 5.

4.2.1. Listeners

Offline ASR model. The offline transcriber is the `faster-whisper` implementation [50] of the Whisper [41] model, run with the `large-v3-turbo` checkpoint. This is Whisper large-v3 with its decoder pruned from 32 layers to 4 and then fine-tuned, giving 809 M parameters against the 1,550 M of large-v3 [51]. The large Whisper variants are a common choice for scoring how intelligible generated and extracted speech is [52, 53]. Decoding was greedy, with a single sampling temperature of zero and no conditioning on previously decoded text, so the same audio always produces the same transcript, and the language was fixed to English. The smaller `small.en` checkpoint (244 M parameters) is used only inside the training loop, where it transcribes a validation sample every epoch to shortlist checkpoints and schedule the learning rate.

Live model. The live listener is `gemini-3.7-flash` [7], which was queried via the `google-genai` Python package through the Application Programming Interface (API). The

live model, like the ASR model, does not separate anything. The Gemini model is prompted to transcribe what it can hear in the extractor's output. The live model is the tool for which this work is being built and so its result is the most important with respect to the overall evaluation of the extractor. The prompt used can be found in Appendix D.1 and is deliberately worded to avoid instructing the live model to perform extraction. The live model is also prompted to provide a structured response with a status field alongside the transcript. The status field is used to indicate whether the live model heard speech or not, and if it did not hear speech, the transcript is left empty. This allows for a clear distinction for whether the model hallucinates as the status field can be used to determine whether the model heard speech.

Why Gemini Flash and not Gemini Live. Gemini Live, although the target deployment of this work, is not used in the development of the extractor. The main research concern is not *when* the model replies, but *what* the model understood from the extraction. Gemini Live receives audio as a stream, decides from pauses whether a target has finished speaking and can interrupt. Since it ends the input when it detects silence, a trial will cut short when the target pauses, or when an interferer is masked. Gemini Live will reply then to a fragment of the trial, and the score would reflect where Live decided the turn ended rather than how well the extractor performed. Gemini 3.7 Flash, however, can receive a whole trial and score against a full transcript. The extractor itself does not depend on which model listens, so Live only provides additional difficulty for few justifiable reasons in the development process. The only benefit in using Live would be to measure turn-taking, but this is not the focus of this work.

Speech gating. The Silero VAD v6.2.1 [54] speech gate is implemented to filter out any audio that appears to contain no speech and acts as a pre-filter for the transcription tools. A clip without detectable speech is scored as an empty transcript without calling the listener. This method is supported by [55] where the authors show that this method reduces inventing words on silence and mirrors what a deployed system would do. The gated output also shows missed words that a target said but were suppressed, and secondly shows correct output of silence when the target is absent.

4.2.2. Latency measures

Latency is measured against a 200–300 ms end-to-end budget. There are two components to consider: time to consume and time to process the audio. The real-time factor (RTF) is the ratio of the time taken to process the audio to the length of the audio, and for a model to be able to keep up with real-time audio, the RTF must be less than 1. The second component considers the latency created by the architecture itself which cannot be improved by faster hardware. To produce the output frame for a given window, the model must first have received that entire window. This forces a delay of the processing of the window as per Section 3.2.2.

5. Results

This section covers the three research questions with results to answer them individually. In short, the signal and perceptual metrics do not rank the systems in the same order as the Gemini LCF-WER. Secondly, cutting leaked words comes with more invented words. The hypothesis, tested in Section 5.2, is that both errors come from how hard the mask suppresses frames the two speakers share. Finally, the speaker-specific information provides the extractor with meaningful knowledge to better isolate the target speaker and still fits in the causal latency budget.

5.1. RQ1: Conventional measures and the listener

The three kinds of separation considered in this project are the total error, interference from other speakers and finally from artefacts introduced by the extractor system. The signal-domain separation results are shown in Figure 5.1. These metrics highlight what the extractor system is doing in the signal domain and can best be used to understand the trade-offs between the different types of separation.

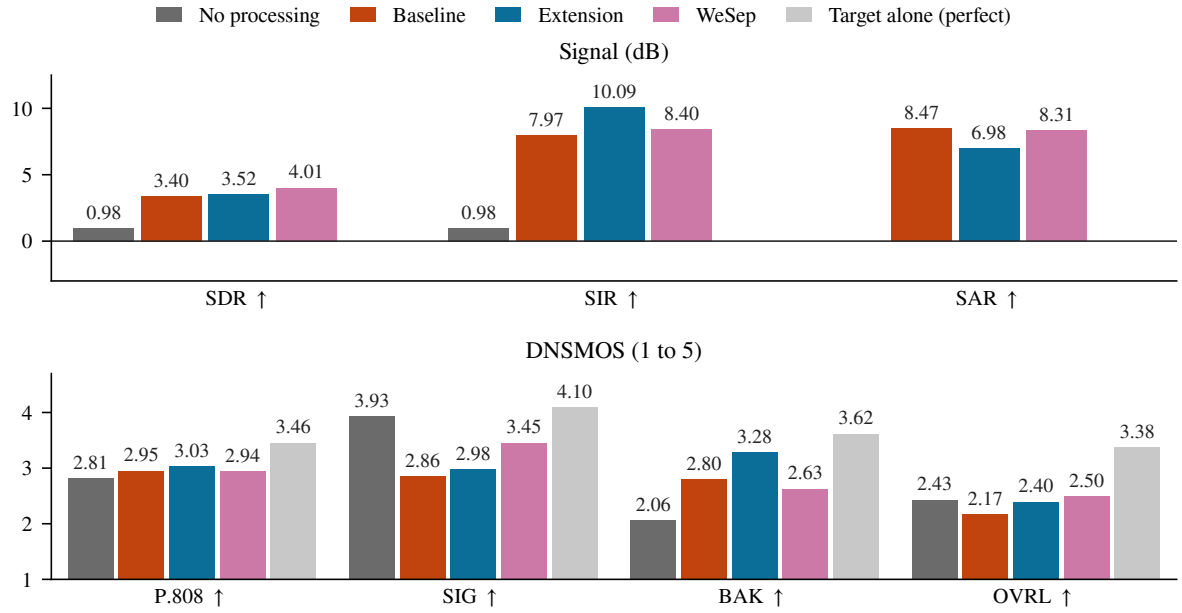


Figure 5.1: Conventional measures on the 249 two-speaker test trials. The signal decomposition and DNSMOS predicted opinion scores (1 to 5). Higher is better for every measure. Bars at the $+30\,\mathrm{dB}$ cap set by the denominator floor $\varepsilon_{\text{pres}} = 10^{-3}$ (Section 3.4.6) are left out, as they are not measurements (AI3).

Table 5.1: Gemini's word error rate (LCF-WER) on the 248 two-speaker test trials, with its 95 % interval in percentage points, redrawing whole speakers (Section 3.4.8). Lower is better.

	No processing	Baseline	Extension	WeSep	Target alone (perfect)
LCF-WER (%) ↓	52.05	48.46	41.81	**34.08**	1.78
95 % CI	[44.81, 60.65]	[42.17, 55.58]	[35.64, 48.72]	[28.74, 40.06]	[1.33, 2.19]

The extension has improved the SIR metric over the baseline, and has beaten the WeSep model (SIR: baseline 7.97, extension 10.09, WeSep 8.40 dB). This means that the extension is able to *remove the interfering speaker's energy the best*. The gap in performance between the extension and WeSep therefore seems to lie in the SAR and SDR metrics (SAR: extension 6.98, WeSep 8.31 dB; SDR: extension 3.52, WeSep 4.01 dB). The baseline has the highest model SAR metric (SAR: baseline 8.47, WeSep 8.31, extension 6.98 dB). This should not be attributed to the model being better at reducing artefacts, but rather to the simplistic nature of the model design. The baseline model is not as aggressive in masking out the interfering speaker's content (SIR: baseline 7.97, extension 10.09 dB), and therefore does not introduce as many artefacts into the output. On P.808 alone, the extension's predicted listener rating is higher than WeSep's (3.03 against 2.94). When considering the other three metrics, the WeSep model has a better SIG and OVRL score than the extension. The extension scores lower than no processing for both the SIG and OVRL metrics.

The WeSep model shows the highest SDR of the models, not by a convincing margin, and has a clear lowest LCF-WER (34.08 %) as seen in Table 5.1. When considering each metric individually, a single signal metric does not explain the better Gemini performance across the models. For example, the extension has better SIR than WeSep, the baseline beats WeSep on SAR, and for SDR the baseline and extension have comparable figures despite the clear ranking set by Gemini. Therefore by the signal metrics alone, WeSep cannot be established as a better separation model, and it does not explain why Gemini performs better on WeSep's output. Next, the perceptual quality metrics are not good indicators for LCF-WER either. DNSMOS predicts how pleasant the audio sounds to people, not how much of the target's speech a listener can recover. Audio that sounds good is not necessarily better for content fidelity. This is established by the extension scoring higher than WeSep on P.808 and BAK, yet having a clearly higher LCF-WER (41.81 against 34.08 %).

5.2. RQ2: Leaked and invented words

The live model results on the 248 two-speaker test trials are shown in Figure 5.2. These results are most indicative of the overall performance of the extractor system as the live model is the end user of the extraction process.

The baseline lowers the LCF-WER by 3.59 points relative to no processing (no processing 52.05, baseline 48.46 %), but the 95 % interval of this change, $[-9.33, +1.97]$, includes zero, so on the test set the baseline cannot be distinguished from passing the mixture unprocessed. It does include less of the interfering speaker's content, as the average percentage of interfering words in the output is 14.67 points lower than with no processing (mean leak: no processing 47.89, baseline 33.22 %). This came at a trade-off with the number of invented words. The baseline system consequently invents more words on average per trial as seen in Figure 5.2 (invented per trial: no processing 1.48, baseline 2.24). The baseline system also more frequently invents words, as indicated by FR@2 (no processing 30.24, baseline 41.13 %). Therefore,

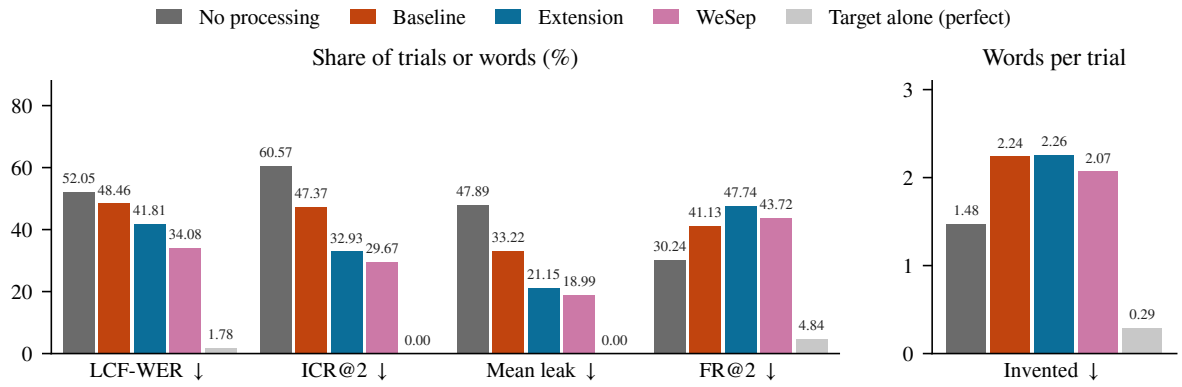


Figure 5.2: Content recovered by Gemini (`gemini-3.7-flash`, audio in and text out) on the 248 two-speaker test trials (AI3). Lower is better for every measure.

both the frequency and quantity of invented words have increased over passing the audio unprocessed. The extension has both reduced the share of leaked words per trial (baseline 33.22, extension 21.15 %) and the frequency of leakage (ICR@2: baseline 47.37, extension 32.93 %) by substantial amounts. Unlike the leakage, the extension does not invent fewer words than the baseline (2.24 against 2.26 words per trial), and it invents two or more words in more trials (FR@2: baseline 41.13, extension 47.74 %). Both invent well above the 1.48 words per trial of no processing. Another important finding is that *all* models invent more than passing the mixture unprocessed (baseline 2.24, extension 2.26, WeSep 2.07), which is consistent with the extractor's artefacts being heard as words.

Table 5.2: Trade-off test on the extension's loss, on the 103 two-speaker validation trials, as the fine-tunes were not scored on the test set. The top section is each run's score. The remaining sections show the change when one weight is added in percentage points for leaked words and LCF-WER, with its 95 % interval from a paired bootstrap.

	Leaked words (%)		Invented per trial		LCF-WER (%)	
	value	95 % CI	value	95 % CI	value	95 % CI
Score of each fine-tune						
Control	21.57		2.10		38.24	
Leak-weighted	18.31		2.82		38.71	
Artefact-weighted	36.23		1.90		47.58	
Both weights	29.84		2.06		42.87	
Change when the leak weight is added to						
control	−3.26	[−6.59, −0.03]	+0.72	[+0.27, +1.21]	+0.47	[−3.20, +4.15]
artefact-weighted	−6.39	[−12.43, −0.51]	+0.16	[−0.28, +0.60]	−4.71	[−9.74, +0.15]
Change when the artefact weight is added to						
control	+14.66	[+9.09, +20.52]	−0.20	[−0.65, +0.25]	+9.34	[+3.65, +15.37]
leak-weighted	+11.53	[+6.19, +17.06]	−0.76	[−1.28, −0.25]	+4.16	[−0.69, +9.41]

Four fine-tunes were run on the extension where 6 additional epochs were trained. As shown in Table 5.2, adding the leak weight reliably cut leaked words, against both the control (-3.26) and the artefact-weighted run (-6.39). Adding the artefact weight cut invented words only on top of the leak weight but against the control, the change is within the noise [-0.65,

$+0.25$]. However, in three of the four cases, this comes at the expense of the other error. Crucially, it is a hypothesis that these errors are not trade-offs of each other, but a by-product of the trade-off of the masking architecture. By weighting by a different error, the masking behaviour of the extractor changes. With reference to the control, none of the fine-tunes improves the LCF-WER score on average, at most performing within the noise of the control as seen for the leak-weighted fine-tune (LCF-WER change from the control: leak weight $+0.47$). Detrimentally, the artefact weight alone clearly worsens the LCF-WER (LCF-WER: control 38.24, artefact-weighted 47.58 %).

The trade-off of the masking architectures. As a masking architecture focuses on removing interfering content, by masking frames that contain the interfering speaker's content, the overlapping frames that contain the target speaker's content are also masked. The hypothesis is that from the perspective of the extractor, a single frame only contains raw audio of unknown mixture quantities of different sources. As a result, the more aggressive the extractor becomes in removing content from a frame, the more likely it is to remove the target speaker's content as well. Therefore, more aggressive interferer suppression possibly leads to a quicker degradation of audio quality, which then is misinterpreted by the listener as invented words. Conversely, the weaker the suppression of the interfering speaker, the more likely that the interferer content is included in the output. The trade-off between the two errors is hypothesised to be a result of the masking procedure and not between the errors themselves.

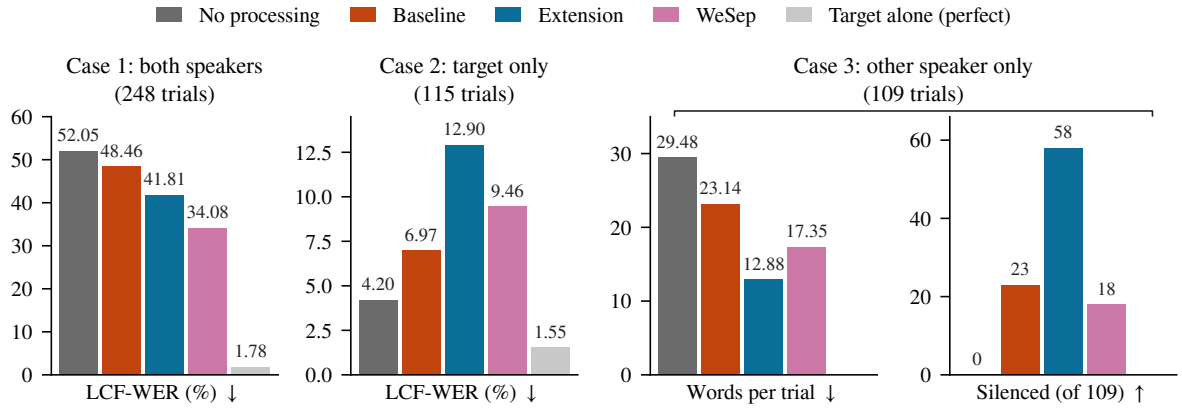


Figure 5.3: Gemini's response in each trial case of Section 3.1.2 on the test set. Cases 1 and 2, where the target spoke, report LCF-WER. Case 3, where only the other speaker spoke, reports the words Gemini wrote per trial, where 0 is correct, and how many outputs the speech gate silenced. Case 4 (noise only, $n = 27$) is omitted, as the speech gate silenced almost every output there: all 27 of the baseline's, 26 of the extension's and 24 of WeSep's (AI3).

Case breakdown of how the extractor performs. Figure 5.3 highlights where the extension shines and where it does not. When only the interfering speaker is present, the extension performs better than WeSep, as Gemini writes fewer words per trial from its output, where 0 is correct (words per trial: baseline 23.14, extension 12.88, WeSep 17.35). Secondly, if the extractor needs to suppress the other speaker such that a voice activity detector (VAD) does not

detect any speech, the extension is able to do this more often than WeSep, getting 58 of the 109 clips silenced as compared against WeSep's 18 (Case 3 outputs silenced). The improvement in the extension's performance may be attributed to the training examples that include those specific cases, which WeSep was not trained on. However, when only the target is present, the extension performs worse than WeSep (Case 2 LCF-WER: extension 12.90, WeSep 9.46 %), and 7 of its 115 target-only outputs were silenced entirely. Interestingly, all the models perform worse on the target-only case than just leaving the target unprocessed (Case 2 LCF-WER: no processing 4.20, baseline 6.97, extension 12.90, WeSep 9.46 %). Without any processing, Gemini already recovers most of the target from the target and the real background noise of the dataset, which leaves the extractor room only to introduce artefacts that are misinterpreted by Gemini. Overall, the extension is better than WeSep at keeping the other speaker out when the target is absent, although it still falls short when the target is present, both with two speakers (Case 1 LCF-WER on the test set: extension 41.81, WeSep 34.08 %) and alone.

5.3. RQ3: Voice information in real time

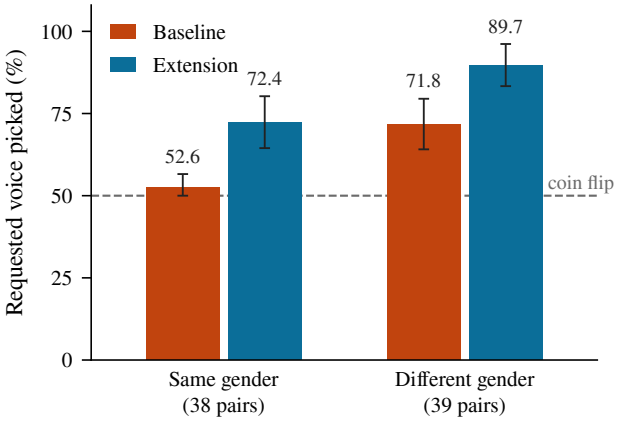


Figure 5.4: How often each extractor returns the voice it is asked for, on the two-voice validation pairs (AI3).

The extension system is a clear improvement over the baseline, which is indicative that the extractor has begun to find a signal in the mixture that is associated with the target speaker, but it must now be considered whether the extension can be used in real-time applications. More specifically, can the extractor better choose the right speaker in the latency budget? This is demonstrated by Figure 5.4, which highlights how even in the difficult case where both speakers are the same gender, the extension is able to pick the right speaker more often than the baseline. When the two speakers differ in gender, their voices sit in different pitch ranges, which is the easier case. When they share a gender, the voices overlap and the extractor must use finer differences. Here the baseline picked the right voice 52.63 % of the time, which cannot be told apart from chance, and the extension 72.37 %, a gain of 19.74 points. The speaker cue change and the ECAPA embedding provide sufficient information to isolate the correct speaker. The latency and throughput results are shown in Figure 5.5. The baseline and the extension are able to meet the real-time factor and latency requirements for a live system (RTF < 1.0).

The WeSep model is not able to meet the requirements and therefore cannot be used in a live system (WeSep RTF 2.85, mean delay 348.32 ms). Furthermore, the extension did not miss any chunks and the worst-case delay was 196 ms, which is below the 200–300 ms requirement.

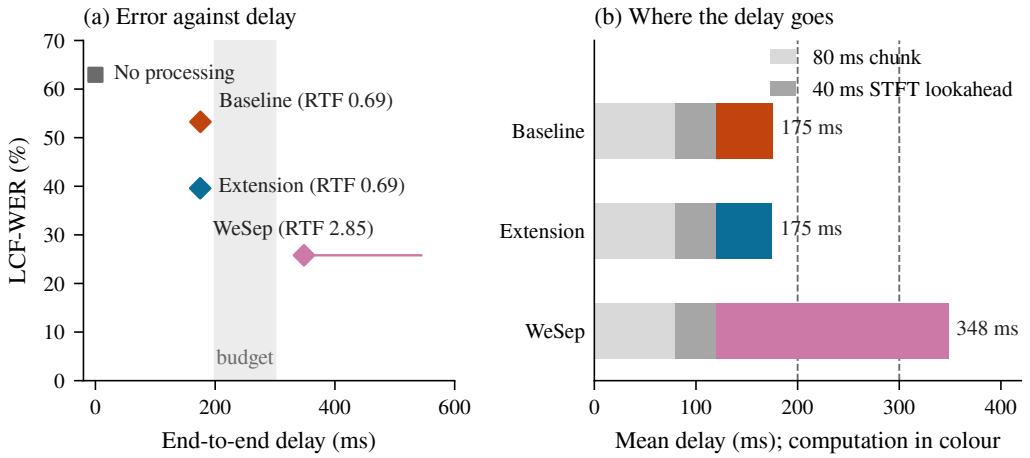


Figure 5.5: Delay and accuracy. (a) LCF-WER on the validation set against mean end-to-end delay with the real-time factor (RTF) in brackets. The delay of a model does not depend on which trials it processes, so the validation set is used here. (b) Mean delay split into the 80 ms chunk, the 40 ms STFT lookahead and computation, with the 200–300 ms budget dashed. Timed on an Intel Core i5-1135G7 laptop CPU (4 cores, 8 threads, 15 GB RAM) over a median of three runs (AI3).

5.4. Unseen and real recordings

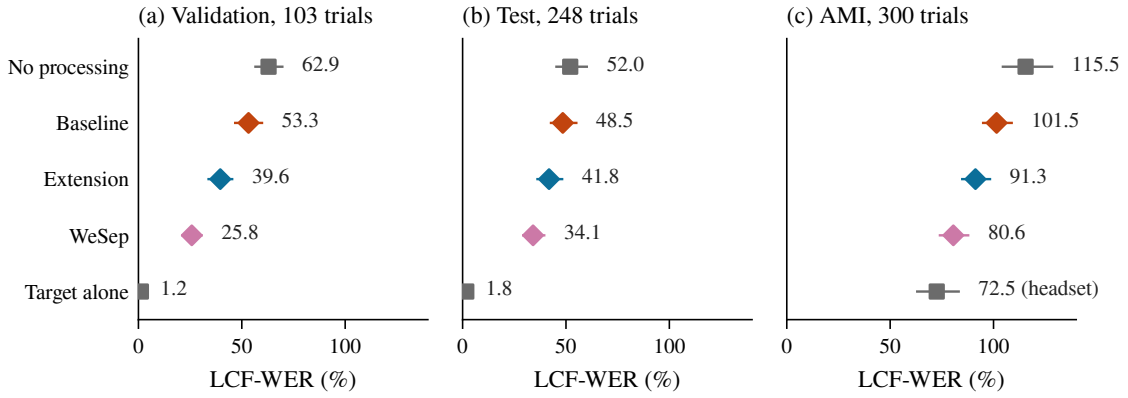


Figure 5.6: Gemini's LCF-WER for each system on the three evaluation sets, where lower is better (AI3).

Table 5.3 shows Gemini's results on the AMI meeting recordings of Appendix B, and Figure 5.6 compares the LCF-WER on the validation, test and AMI sets. The conclusion is the same as for the simulated data: the extension improves the LCF-WER over doing nothing, although the offline WeSep system performs better for the strict extraction task. Of the extractors, WeSep has the lowest LCF-WER (80.56 %), followed by the extension (91.26 %), the baseline (101.50 %) and then no processing (115.51 %). A demonstration of how the model distorts the speech can be seen by the target words missed metric, where Gemini is able without any processing to only miss 3.46 % of the target words, but even with WeSep, Gemini misses 27.71 % of the

Table 5.3: Content recovered by Gemini on 300 real two-speaker stretches of AMI meeting recordings (Appendix B). Lower is better in every row.

	No processing	Baseline	Extension	WeSep	Target's headset
target words missed (%) ↓	3.46	**27.33**	45.25	27.71	2.31
ICR@2 (%) ↓	89.14	44.19	33.71	**27.72**	58.80
mean leak (%) ↓	82.80	31.63	25.19	**19.64**	51.96
invented /trial ↓	1.48	3.35	**1.83**	2.65	1.13
FR@2 (%) ↓	38.33	65.38	**49.46**	64.17	30.10
silenced (of 300) ↓	0	58	110	**38**	0

target words. This indicates how the extractor removes the background interfering speech at the cost of degrading the target speech. Finally, there is an increase in artefacts introduced by the extractor, as seen in the increase in invented words per trial (1.48 to 1.83) and rate of fabrication (FR@2, 38.33 % to 49.46 %) for the extension, which is an expected result from the simulated dataset.

The extension has been shown to beat the causal re-implementation in this work of the baseline of the REAL-TSE model, in which additional speaker information can be added without impacting the real-time factor or latency of the system. This improvement holds on both the simulated and real-world data. On AMI, the extension lowers both the share of leaked words and the number of invented words compared to the baseline (mean leak: baseline 31.63, extension 25.19 %; invented per trial: baseline 3.35, extension 1.83), and even beats WeSep on invented words (invented per trial: extension 1.83, WeSep 2.65).

6. Conclusion

The work in this report set out to build a target speaker extractor that could prepare audio for a live speech-to-speech model under the constraints of being able to run in real time, causally and on a single channel of audio. The extractor was judged by what Gemini recovers from the extracted output, and not by the actual separation quality of target speech from the other audio. All of the findings relate to at most two-speaker mixtures, and are optimised and tested for Gemini.

Key findings. The general findings include how perceptual and signal domain measures do not align with the live model's ability to transcribe the audio. Right now the metrics act as a proxy for how the model extracts, although not perfectly, and do not necessarily have a direct correlation with how well the extractor performs for the downstream model **(RQ1)**. The simple word error rate hides the trade-off that matters for the downstream model. The word error rate alone does not discern how insertions or substitutions are obtained, and both are rooted in the architecture. These errors are either invented or leaked words, and both are hypothesised to come from the level of aggressiveness of the masking architecture **(RQ2)**. The next finding is that telling the extractor more about the target's voice is what helped the most. This involved considering the enrollment and the conditioning of the extractor on the target

speaker's embedding. Splitting the cue into parts and adding the speaker embedding lowered Gemini's word error rate on the test set from 48.46 % for the baseline to 41.81 % **(RQ3)**.

Limitations. The dataset is restricted to two speakers, which can confound the extractor to expect the target to be present if two speakers are present. Secondly, the combined audio forms *read* speech and not *conversational* speech. This means that the resulting audio does not have natural turn-taking, or conversational dynamics. These dynamics are simulated through the overlap of the target and interferer, but it is not a perfect simulation. Next, the dataset is restricted to English speech, and shoebox rooms only. Another limitation is that Gemini can change as it is a closed model both between runs and versions.

Future work. The first suggestion is considering the streaming architecture itself. The LSTM layers of the extractor could be replaced with state space layers. A state space layer is a recurrent layer whose updates are linear, which allows for parallel training while still running frame by frame. SA-Mamba [56] applies this idea within a band-split, dual-path design similar to the one in this work, and conditions the state space layers on the target speaker. It placed second in the online track of the REAL-TSE challenge, while its report describes training on simulated mixtures only, and the organisers attribute most of the gains of the top systems to their data [9]. The architecture has not yet been used with a live speech model as the listener and secondly it did not have a training recipe built around one, as in this work. The next suggestion would be to consider the current objective function. It is a proxy for general extraction rather than for the downstream task. Weighting the leaked speech or the artefacts in the loss moved errors from one source to the other but did not lower the LCF-WER (Section 5.2), so an objective closer to *how* Gemini listens is still needed. Finally, the presence gate was explored only briefly in this work, and only trained with the extension. The gate should next be trained on the two-speaker condition rather than a general target-present condition. Right now the main issue is the discernment of how much to mask the interferer. The target presence gate was just a first step. It estimates for every frame the probability that the target is present, and the Markov prior calculates how likely the target is to continue speaking. A natural next step is a gate that, when a frame is shared by both speakers, rather than just a target being present, estimates how much of the frame belongs to the target and how much belongs to the interferer. Then the extractor could follow overlapping speech more closely and remove only what is required.

Plagiarism and AI declaration

During the course of this project, I made use of Claude Code to assist me in the research process. It was used in general cases where redundant tasks / code was needed (soft use) and then cases where a citation for the tool is required (hard use) in accordance with the guidelines set out in "Guidelines for using GenAI tools in Learning and Assessment". Listed below are all the specific items that I used the tool for:

AI1. **Research:** I prompted Claude Code to assist me in finding relevant research papers and articles that may have potentially solved the TSE task. I used the tool to highlight shortcomings and advantages of different approaches. It was from this research that I decided on the BSRNN model. Specifically for the extension of the BSRNN model, I prompted the tool to analyse the shortcomings of the model and to suggest potential avenues to explore to improve the model. I then decided on the specific design choices for the extension from my own understanding of the tool's response.

AI2. **Writing assistant:** I wrote the report myself and made use of Claude Code in fixing grammar, spelling and sentence structure. The tool was not given access to writing directly to the report, but rather would suggest corrections if a claim or sentence was not clear or inaccurate. I would ask the tool if my summary of a paper or method was fair and accurate to not misrepresent the work of others or my own work. The formulations and equations for the described methods were sourced by the original authors, but I would first rewrite them from my understanding and then ask the tool to help change the notation to be consistent with the rest of my report. Finally, if there was a claim that needed justification, I used the tool to find a relevant paper to support the claim.

AI3. **Diagrams and plots:** Figure 3.1 specifically I prompted Claude Code to assist with the TikZ code for the best layout of the diagram. The other figures in the report were reformatted and edited by Claude Code to have a scientific style and to be consistent with the rest of the report. For the plots, their layout and style was suggested by the tool.

AI4. **Loss function:** The terms in the objective function (Section 3.2.9) were sourced from CARTSE [1] and Yu et al. [14] where explicitly stated. However, I made use of Claude Code for suggestions on how to weight the terms such that one term did not dominate the others.

AI5. **Code (soft):** I made use of Claude Code to assist me in documenting the decisions from my discussion with the tool into the code and recorded them separately. All decisions were logged into the documentation. For the code pertinent to the baseline and extension model for the actual TSE task, I made use of Claude Code to debug and fix errors. Furthermore, for the baseline, because it needed to follow the architecture found in the REAL-TSE paper, I needed assistance in implementing the architecture as described in the paper. I

prompted the tool to verify that the implementation was consistent with the paper and to suggest changes if it was not. Finally, during the course of the research, I discussed potential avenues to explore in which the tool would make experiments to test whether the idea was worth pursuing.

AI6. **Code (hard):** I originally wrote my code in scripts but Kaggle only accepts the files in a notebook format. Thus I prompted the tool to fully convert my scripts into a bundle of configuration files and notebooks that can be run on Kaggle. The tool then created multiple hyperparameter configuration files (*.yaml*), notebooks for training different models and dataset loaders. I then prompted the tool to verify that the notebooks were consistent with the scripts and that they would run on Kaggle. The generated files only contain the code required to train and save the models and no additional logic is used to change the model or training procedure. These are files and functions that do not form part of the core logic, but rather for setup, configuration, tests, or redundant tasks.

AI7. **Equations and Formulations:**

(a) For the first extension, the idea of splitting the cue into parts came after the initial diagnosis of the failures of the BSRNN. I prompted Claude Code with ideas for improving the speaker representation in which the splitting of the cue into parts was suggested. I used Claude to verify the equations and to ensure consistent notation.

(b) I had the idea of using a two-state target-present Markov model as a conditioning gate for the BSRNN model. I was able to derive the expressions by hand until the KL term myself. For the KL term, Claude Code pointed out that under the mean-field assumption the expectation over the neighbouring frames splits into the product of their probabilities and it cited this identity to the paper by Ghahramani and Jordan [37]. I then worked through each step myself. For the training and inference, Claude Code suggested and derived the equations for the Monte Carlo sampling and the binary Concrete distribution.

AI8. **Created Metrics:** The new metrics, the fabrication rate and the interferer content rate, were my own ideas, in which I only used Claude to help formulate the equations.

References

[1] L. Li and S. Seki, "CARTSE submission to the REAL-TSE challenge track 1: Online target speaker extraction," CyberAgent, Inc., Tech. Rep., 2026, REAL-TSE Challenge Track 1 system description. [Online]. Available: https://real-tse.github.io/assets/pdf/CARTSE-Track1.pdf

[2] Google DeepMind, "Gemini Live API," https://ai.google.dev/gemini-api/docs/live, 2026, accessed: 2026-08-31.

[3] P. Manakul, W. H. Gan, M. J. Ryan, A. S. Khan, W. Sirichotedumrong, K. Pipatanakul, W. Held, and D. Yang, "AudioJudge: Understanding what works in large audio model based speech evaluation," in *Proc. 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2026, pp. 3644–3663.

[4] K. Iwamoto, T. Ochiai, M. Delcroix, R. Ikeshita, H. Sato, S. Araki, and S. Katagiri, "How bad are artifacts?: Analyzing the impact of speech enhancement errors on ASR," in *Proc. Interspeech*, 2022, pp. 5418–5422.

[5] S. Chondhekar, V. Murukuri, R. Vasani, S. Goyal, R. Badami, A. Rana, S. SN, K. Pandia, S. Katiyar, N. Jagadeesh, and S. Gulati, "When de-noising hurts: A systematic study of speech enhancement effects on modern medical ASR systems," arXiv:2512.17562, 2025.

[6] K. Žmolíková, M. Delcroix, T. Ochiai, K. Kinoshita, J. Černocký, and D. Yu, "Neural target speech extraction: An overview," *IEEE Signal Processing Magazine*, vol. 40, no. 3, pp. 8–29, 2023.

[7] Google DeepMind, "Gemini 3.7 Flash," https://ai.google.dev/gemini-api/docs/models, 2026, model `gemini-3.7-flash`. Queried 2026-09-02 to 2026-10-18.

[8] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "Librispeech: An ASR corpus based on public domain audio books," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. South Brisbane, QLD, Australia: IEEE, 2015, pp. 5206–5210.

[9] S. Wang, Z. Qian, K. Zhang, J. Han, Z. Liu, X. Yu, H. Li, M. Delcroix, K. Yu, L. Xie, M. Li, and H. Li, "SLT 2026 REAL-TSE challenge: Real-world target speaker extraction from conversational recordings," arXiv:2607.15198, 2026.

[10] J. Carletta, S. Ashby, S. Bourban, M. Flynn, M. Guillemot, T. Hain, J. Kadlec, V. Karaiskos, W. Kraaij, M. Kronenthal, G. Lathoud, M. Lincoln, A. Lisowska, I. McCowan, W. Post, D. Reidsma, and P. Wellner, "The AMI meeting corpus: A pre-announcement," in *Machine Learning for Multimodal Interaction (MLMI)*, ser. Lecture Notes in Computer Science, vol. 3869. Springer, 2005, pp. 28–39.

[11] E. C. Cherry, "Some experiments on the recognition of speech, with one and with two ears," *The Journal of the Acoustical Society of America*, vol. 25, no. 5, pp. 975–979, 1953.

[12] Y. Luo and N. Mesgarani, "Conv-TasNet: Surpassing ideal time–frequency magnitude masking for speech separation," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 27, no. 8, pp. 1256–1266, 2019.

[13] Y. Luo and J. Yu, "Music source separation with Band-Split RNN," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 1893–1901, 2023.

[14] J. Yu, H. Chen, Y. Luo, R. Gu, and C. Weng, "High fidelity speech enhancement with Band-split RNN," in *Proc. Interspeech*, 2023, pp. 2483–2487.

[15] J. Cosentino, M. Pariente, S. Cornell, A. Deleforge, and E. Vincent, "LibriMix: An open-source dataset for generalizable speech separation," arXiv:2005.11262, 2020.

[16] J. R. Hershey, Z. Chen, J. Le Roux, and S. Watanabe, "Deep clustering: Discriminative embeddings for segmentation and separation," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2016, pp. 31–35.

[17] W. Zhang, J. Shi, C. Li, S. Watanabe, and Y. Qian, "Closing the gap between time-domain multi-channel speech enhancement on real and simulation conditions," in *Proc. IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 2021, pp. 146–150.

[18] C. K. A. Reddy, V. Gopal, and R. Cutler, "DNSMOS: A non-intrusive perceptual objective speech quality metric to evaluate noise suppressors," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021, pp. 6493–6497.

[19] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023, pp. 46 595–46 623.

[20] Q. Yang, J. Xu, W. Liu, Y. Chu, Z. Jiang, X. Zhou, Y. Leng, Y. Lv, Z. Zhao, C. Zhou, and J. Zhou, "AIR-Bench: Benchmarking large audio-language models via generative comprehension," in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024, pp. 1979–1998.

[21] L. H.-Y. Foo, C.-K. Yang, C.-A. Li, K.-H. Lu, and H.-y. Lee, "All that glitters is not audio: Rethinking text priors and audio reliance in audio-language evaluation," arXiv:2604.24401, 2026, preprint.

[22] A. Koenecke, A. S. G. Choi, K. X. Mei, H. Schellmann, and M. Sloane, "Careless Whisper: Speech-to-text hallucination harms," in *Proc. 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*. ACM, 2024, pp. 1672–1681.

[23] R. F. Fela and P. Mowlaee, "Perceptually better, semantically worse: Measuring speech enhancement impact on LLM-based voice systems," arXiv:2608.30348, 2026, accepted at EMNLP 2026.

[24] G. Wichern, J. Antognini, M. Flynn, L. R. Zhu, E. McQuinn, D. Crow, E. Manilow, and J. Le Roux, "WHAM!: Extending speech separation to noisy environments," in *Proc. Interspeech*, 2019, pp. 1368–1372.

[25] R. Scheibler, E. Bezzam, and I. Dokmanić, "Pyroomacoustics: A python package for audio room simulation and array processing algorithms," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018, pp. 351–355, version 0.10.1.

[26] W. C. Sabine, *Collected Papers on Acoustics*. Cambridge, MA: Harvard University Press, 1922.

[27] S. Li, S. Wang, J. Han, K. Zhang, W. Wang, and H. Li, "REAL-T: Real conversational mixtures for target speaker extraction," in *Proc. Interspeech*, 2025, pp. 1923–1927.

[28] K. Zhang, J. Li, S. Wang, Y. Wei, Y. Wang, Y. Wang, and H. Li, "Multi-level speaker representation for target speaker extraction," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025, pp. 1–5.

[29] B. Desplanques, J. Thienpondt, and K. Demuynck, "ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification," in *Proc. Interspeech*, 2020, pp. 3830–3834.

[30] M. Ravanelli, T. Parcollet, P. Plantinga, A. Rouhe, S. Cornell *et al.*, "SpeechBrain: A general-purpose speech toolkit," arXiv:2106.04624, 2021.

[31] A. Nagrani, J. S. Chung, and A. Zisserman, "VoxCeleb: A large-scale speaker identification dataset," in *Proc. Interspeech*, 2017, pp. 2616–2620.

[32] J. S. Chung, A. Nagrani, and A. Zisserman, "VoxCeleb2: Deep speaker recognition," in *Proc. Interspeech*, 2018, pp. 1086–1090.

[33] S. Wang, K. Zhang, S. Lin, J. Li, X. Wang, M. Ge, J. Yu, Y. Qian, and H. Li, "WeSep: A scalable and flexible toolkit towards generalizable target speaker extraction," in *Proc. Interspeech*, 2024, pp. 4273–4277.

[34] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. International Conference on Learning Representations (ICLR)*, 2014, arXiv:1312.6114.

[35] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989.

[36] D. P. Kingma, S. Mohamed, D. J. Rezende, and M. Welling, "Semi-supervised learning with deep generative models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014, pp. 3581–3589.

[37] Z. Ghahramani and M. I. Jordan, "Factorial hidden Markov models," *Machine Learning*, vol. 29, no. 2–3, pp. 245–273, 1997.

[38] C. J. Maddison, A. Mnih, and Y. W. Teh, "The Concrete distribution: A continuous relaxation of discrete random variables," in *Proc. International Conference on Learning Representations (ICLR)*, 2017, arXiv:1611.00712.

[39] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with Gumbel-Softmax," in *Proc. International Conference on Learning Representations (ICLR)*, 2017, arXiv:1611.01144.

[40] whisper-normalizer contributors, "whisper-normalizer: Standalone text normalisation from OpenAI Whisper," https://pypi.org/project/whisper-normalizer/, 2026, version 0.1.15. Accessed: 2026-08-31.

[41] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. 40th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 28 492–28 518.

[42] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O'Reilly Media, 2009.

[43] C. K. A. Reddy, V. Gopal, and R. Cutler, "DNSMOS P.835: A non-intrusive perceptual objective speech quality metric to evaluate noise suppressors," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 886–890.

[44] International Telecommunication Union, "Subjective test methodology for evaluating speech communication systems that include noise suppression algorithm," ITU-T, Geneva, Recommendation P.835, Nov. 2003.

[45] ——, "Subjective evaluation of speech quality with a crowdsourcing approach," ITU-T, Geneva, Recommendation P.808, Jun. 2018.

[46] E. Vincent, R. Gribonval, and C. Févotte, "Performance measurement in blind audio source separation," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 14, no. 4, pp. 1462–1469, 2006.

[47] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[48] M. Bisani and H. Ney, "Bootstrap estimates for confidence intervals in ASR performance evaluation," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, vol. 1, 2004, pp. 409–412.

[49] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. International Conference on Learning Representations (ICLR)*, 2019, arXiv:1711.05101.

[50] SYSTRAN, "faster-whisper: Faster Whisper transcription with CTranslate2," https://github.com/SYSTRAN/faster-whisper, 2026, version 1.2.1. Accessed: 2026-08-31.

[51] OpenAI, "whisper-large-v3-turbo," https://huggingface.co/openai/whisper-large-v3-turbo, 2024, model card. Accessed: 2026-09-28.

[52] P. Anastassiou *et al.*, "Seed-TTS: A family of high-quality versatile speech generation models," arXiv:2406.02430, 2024.

[53] H. Wang, J. Hai, D. Yang, C. Chen, K. Li, J. Peng, T. Thebaud, L. Moro-Velázquez, J. Villalba, and N. Dehak, "SoloSpeech: Enhancing intelligibility and quality in target speech extraction through a cascaded generative pipeline," arXiv:2505.19314, 2025.

[54] Silero Team, "Silero VAD: Pre-trained enterprise-grade voice activity detector," https://github.com/snakers4/silero-vad, 2021, version 6.2.1.

[55] M. Barański, J. Jasiński, J. Bartolewska, S. Kacprzak, M. Witkowski, and K. Kowalczyk, "Investigation of Whisper ASR hallucinations induced by non-speech audio," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025, pp. 1–5.

[56] B. Liu, "Speaker-aware state space modeling for streaming target speaker extraction," DeepSound, Tech. Rep., 2026, REAL-TSE Challenge Track 1 system description. [Online]. Available: https://real-tse.github.io/assets/pdf/DeepSound-Track1.pdf

[57] jiwer contributors, "jiwer: Evaluate automatic speech recognition," https://github.com/jitsi/jiwer, 2026, version 4.0.0. Accessed: 2026-08-31.

A. Construction parameters

Every parameter below is drawn independently per trial from the stated distribution, except where a regime overrides it. The table gives the training and validation settings. The `eval_public` and `eval_private` splits draw SIR from $[-5, 15]$ dB and use no regimes. Ranges written $[a, b]$ are sampled uniformly on that interval.

Parameter	Distribution	Meaning
Trial layout		
Mixture length	$[15.0, 20.0]$ s	Duration of the rendered trial
Overlap ratio	$[0.0, 0.7]$	Fraction of target speech with the interferer also talking
Overlap tolerance	0.03	Accepted deviation from the requested overlap
Target activity ratio	$[0.15, 0.78]$	Fraction of the trial in which the target speaks
Activity tolerance	0.1	Accepted deviation from the requested activity
Utterances per source	≤ 6	Cap on how many utterances one recording may contribute
Levels		
SIR	$[-10.0, 10.0]$ dB	Target loudness relative to the interferer
SNR	$[0.0, 20.0]$ dB	Target loudness relative to the noise bed
Target loudness	$[-33.0, -25.0]$ LUFS	Absolute level of the target
Clip ceiling	0.95	Peak after which the whole trial is rescaled together
Room		
T_{60}	$[0.25, 0.6]$ s	Reverberation time
Room length, width	$[5.0, 10.0]$ m	Floor plan
Room height	$[3.0, 4.0]$ m	
Source distance	$[0.1, 2.0]$ m	Talker to microphone
Source height	$[0.9, 1.8]$ m	Talker, seated to standing
Microphone height	$[0.9, 1.8]$ m	
Early reflections	50 ms	Cutoff separating early echoes from the late tail
Enrollment		
Enrollment length	5.0 s	Single clip, levelled, carrying no room
EQ augmentation	$p = 0.5$	Three peaking biquads, log-uniform 120 Hz to 6400 Hz, ± 6 dB, $Q = 1.0$, RMS restored

Continued on the next page

Parameter	Distribution	Meaning
Speaker pairing		
Same-gender fraction	0.5	Proportion of trials whose two talkers share a gender
Speech detection [54]		
Silence threshold	250 ms	Pause length after which a talker counts as stopped
Speech probability	0.5	Frame-level threshold
Minimum speech	100 ms	Shortest run counted as speech
Speech padding	30 ms	Added either side of a detected run
Noise-bed rejection	0.5 s	Longest run of detected speech tolerated in a WHAM! clip
Regimes		
Regime weights	0.6 / 0.4	Proportion of trials drawn as `base` / `hard`
`base` SNR	$[8.0, 20.0]$ dB	Regime override
`base` activity	$[0.45, 0.78]$	Regime override
`base` T_{60}	$[0.25, 0.5]$ s	Regime override

Table A.1: Construction parameters and their sampling distributions.

B. AMI evaluation set

Property	Value
Recordings	
Source	AMI Meeting Corpus, scenario meetings [10]
Trials	300, totalling 77.9 min
Trial length	Median 15.8 s, range 10.1–20.0 s
Rooms	3, one per recording site (Edinburgh, IDIAP, TNO)
Meetings	58
Model input	One table microphone, level as recorded
Target alone	The target's headset and it also picks up the other speaker
Enrollment	5.0 s of the target speaking alone, from their headset in a different meeting; 247 distinct clips
Speakers	

Continued on the next page

Property	Value
Unique speakers	60
Trials per target	Median 5, range 1–12
Sex	45 male, 15 female
Pairing	236 same-sex trials, 64 mixed-sex
Native English	21 of 60 speakers; 108 of 300 trials have a native-English target
Age	Mean 26.8, median 25.5, range 19–58 years
Content	
Target words	Median 14 per trial, range 5–52
Other speaker's words	Median 15.5 per trial, range 5–70

Table B.1: The AMI evaluation set.

C. Presence gate

C.1. Markov prior

The transition matrix of the Markov prior in (3.20), set by (3.23). Rows are the state in frame $t-1$ and columns the state in frame t, with 0 for the target silent and 1 for the target speaking:

$$A = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} = \begin{bmatrix} 1 - 1/\bar{d}_{\text{off}} & 1/\bar{d}_{\text{off}} \\ 1/\bar{d}_{\text{on}} & 1 - 1/\bar{d}_{\text{on}} \end{bmatrix} = \begin{bmatrix} 0.99796 & 0.00204 \\ 0.00168 & 0.99832 \end{bmatrix}, \tag{C.1}$$

with $\bar{d}_{\text{on}} = 596.34$ frames ($4.77\,\text{s}$) and $\bar{d}_{\text{off}} = 489.53$ frames ($3.92\,\text{s}$), measured over the 9955 training trials with each speaker in turn as the target, as in training.

D. Evaluation protocol details

D.1. Prompt for live-model transcription

```
Write down the words spoken in this audio, exactly as spoken.

If no words are audible, leave the transcript empty and set status to no_speech.
```

D.2. Pinned components of the measuring instrument

Component	Identity
Text normaliser	`whisper-normalizer` 0.1.15, `EnglishTextNormalizer` [40, 41]
Baseline transcriber	`faster-whisper` 1.2.1, `large-v3-turbo`, int8 quantisation, greedy decoding [41, 50, 51]
In-loop transcriber	`faster-whisper` 1.2.1, `small.en`, int8 quantisation, greedy decoding
Word error rate	`jiwer` 4.0.0 [57]
Stopword list	`NLTK` English stopwords [42]
Live model	`gemini-3.7-flash`
Random seed	42

Table D.1: Components of the evaluation instrument.